Combinatorics on Words

Suppression of Unfavorable Factors in Pattern Avoidance

Veikko Keränen

We explain extensive computer-aided searches that have been carried out for many years to find new ways of constructing abelian square-free words over four letters. These structures have turned out to be very rare and hard to find. We also encountered highly nonlinear phenomena that considerably affected our calculations and usually made them hard to accomplish. However, quite recently, we gained new insight into why these structures are so very rare. Consequently, the present work has the potential to make future explorations easier. The rarity of long words that avoid abelian squares can be explained, at least partly, by using the concept of an unfavorable factor. The purpose of this article is to describe the use of *Mathematica* in searching for and suppressing these factors. In principle, the same programs can be used with slight modifications for other kinds of word patterns as well.

■ Introduction

An abelian square is a nonempty word uv, where u and v are permutations (anagrams) of each other. In 1961, Paul Erdös [1] raised the question of whether or not abelian squares can be avoided (as factors) in infinitely long words (also called strings). In 1969, Pleasants [2] solved positively the question by Erdös in the case of a five-letter alphabet, but the four-letter case remained open until 1992 when the author [3] presented an abelian square-free (a-2-free) endomorphism g_{85} over the four-letter alphabet $\Sigma_4 = \{a, b, c, d\}$. This endomorphism g_{85} was found after long computer experiments, and, for a long time, all known methods for constructing arbitrarily long a-2-free words on Σ_4 were based on the structure of g_{85}, including Carpi's [4] modification from 1998.

In 2002, after over 11 years of exhaustive searches, we found a completely new endomorphism g_{98} of Σ_4^*, the iteration of which produces an infinite a-2-free word. The endomorphism g_{98} is not an a-2-free endomorphism itself, since it does not preserve the a-2-freeness of all words of length 7. However, g_{98} can be used with g_{85} to produce a-2-free DT0L-languages of unlimited size. For the latest remarkable development, the reader is referred to the author's publications [5, 6, 7]. Nowadays, a-2-free words have also found their way to applications, for instance, in number theory, algorithmic music, and only recently in cryptography (Rivest [8] in 2005, Andreeva et al. [9] in 2008).

An important analogous open avoidability problem for the three-letter case was posed by Sami Mäkelä [10] in 2002. In this problem, shortest possible abelian

squares, that is, repetitions of the form xx or xxx for a letter $x \in \Sigma_3 = \{a, b, c\}$, are allowed to occur.

Quite recently we have gained new insight into why these structures are so very rare. We are now able to explain, at least partly, the rarity of long words that avoid abelian squares by using the concept of an unfavorable factor. The purpose of this article is to describe the use of *Mathematica* in searching for and suppressing these factors. In principle, the same programs can be used with slight modifications for other kinds of (possibly nonabelian) word patterns as well.

The concept of an unfavorable factor, together with our programs, can be used in connection with efficient matrix methods presented by Ochem and Reix [11] to find upper bounds for the growth of the number of repetition-free words. Indeed, their approach is also directly applicable to the abelian square-freeness case. The related computations are still proceeding, and, consequently, this topic is not elaborated here.

We have carefully studied the options to make the programs efficient with regard to running time and the required memory space. At the same time, for further development, the structures need to be flexible. Indeed, it is expected that these programs will be run interactively for quite a long time. The programs are also likely to be transformed into various computational environments (such as GRID, C++, FPGA). This work has already been started. In the process of these computations, efficiency will increase due to the natural reduction process. Indeed, the long lists of words that we now have to use can most likely be considerably reduced as one moves on to study longer words.

The programs and some of the structures involved are quite complicated and would have been very hard to develop without using *Mathematica*. We have been using integer coding and cumulative integer lists for words incorporated into quite extensive precomputations. Moreover, certain symbolic representations for words and *Mathematica*'s pattern matching properties (for lists and strings) have been extremely useful.

At the same time, we feel that it would be extremely beneficial for programmability and computational efficiency if in *Mathematica* there were an efficient way of saying, for example, that when matching the pattern `{___,u__,v__}`, we are actually interested only in those cases of `u` and `v` in which their lengths are equal. Indeed, our words can contain thousands of letters, and the absence of restricted pattern matching led us to write part of the programs in a quite tedious C-like fashion. Fortunately, to our big relief, debugging turned out to be a comfortable process. In the future, the new integrated string functions of *Mathematica* like `RegularExpression` can also be used.

We define an *unfavorable* (or *forbidden*) word or factor to be an a-2-free word over a fixed alphabet Σ (in our case $\Sigma = \Sigma_4 = \{a, b, c, d\}$, or $\Sigma = \Sigma_3 = \{a, b, c\}$, in which case we allow xx or xxx for a letter x) if it cannot occur as a proper factor inside any infinite a-2-free word. That is to say, over Σ, an unfavorable a-2-free word cannot be continued to an infinitely long word to the left and to the right without necessarily creating an abelian square at some point. (However, it might well be possible to extend such a word boundlessly in one direction, say to the right, without producing any abelian squares. Experiments support this conjecture, but the existence of such unfavorable factors remains an open question.)

We now give a short explanation of our search for unfavorable factors. Let the alphabet Σ be fixed and consider words over it. We take a word (we actually need to consider all the a-2-free words of a given length) and try to extend it in an a-2-free fashion to the right and left in all possible ways up to a given upper bound for the total length. Each time, the length of the word increases only by a given fixed length. We extend alternately to the right and left, and backtrack when necessary. If the upper bounds are reached, then the original word is *so-far-favorable* (it may still turn out to be unfavorable after more experiments). If there is no way to reach the upper bounds, then the original word is classified, without any doubt, as unfavorable. Thus, for a given length, we obtain three kinds of words: *unfavorable* (*bad*), *so-far-favorable*(*so-far-so-good*), and *favorable* (*good*). The latter type contains words occurring as factors in a-2-free words obtained by using, for example, g_{85}, g_{98}, and Carpi's modification of g_{85}.

It is a remarkable phenomenon that relatively short so-far-favorable words turn out to be unfavorable factors after being "safely" extendable (to the right and left) for a long distance (and with a huge number of branches). One might have expected the long buffers to be sufficient for further growth. Due to Carpi [4] and Keränen [7], we know that the number of a-2-free words over four letters grows exponentially (greater than or equal to 1.02306^n) with respect to word length n. But how do these words grow? We conjecture that in spite of the exponential growth, the ratio between the number of properly extendable, that is, favorable, words of length n, and the number of unfavorable factors of the same length, tends to zero as the length n tends to infinity! Thus, we suspect that the vast majority of a-2-free words over four (and, arguably, three) letters cannot occur as proper factors in the middle of very long (infinite) words. In a way, this would also explain why it is so extremely difficult to find a-2-free endomorphisms over four letters. At present we know, for example, that just a little over half (50.5737%) of the a-2-free words over Σ_4 of length 20 are indeed unfavorable. In the future, this and other similar observations could lead to a better understanding of the abelian square-free structures.

As an example, in the case of four letters, the following words together with their permutations and mirror images are unfavorable (forbidden) factors.

```
In[152]:= unfavourableFactors0fLength8 =
          {"abacdaba","abacdbab","abcabdab","abcabdcb","abcadbad"};

In[153]:= someUnfavourableFactors0fLength20 =
          {"abacabadbacbcdbacdbd","abacabdcadcdbcbacbdc","abacadcabcbdcabdcda
          d","abacdabcbdbcadabdacd",
          "abcabadbcadbdcbcabcd","abcabadcbadbdcbcabcd","abcacdabcdadbabcabda
          ","abcacdabdcadbabcabda",
          "abcacdbacdadbabcabda","abcacdcbabdcdacdcbad","abcbabcdcacdbdabacbc
          ","abcbabdabcacdabcdadb",
          "abcbadbcacdcabdbcdba","abcbadcbdcdacabcadcb","abcbdbcbacbdcdacbdac
          ","abcdbdadcbcadabdadcb"};
```

These words form a very interesting case, leading to a million-fold fluctuational phenomenon (with respect to the required running time and memory space, if we wish to see the tree of all possibilities). This, actually quite frequently appearing, case is illustrated (in a somewhat different setting than the one we use elsewhere in this article) online at [12]. An example is the following illustration for

`"abcbdbcbacbdcdacbdac"`. Note the top at (82, 84 218) and the death of all branches at word length 87.

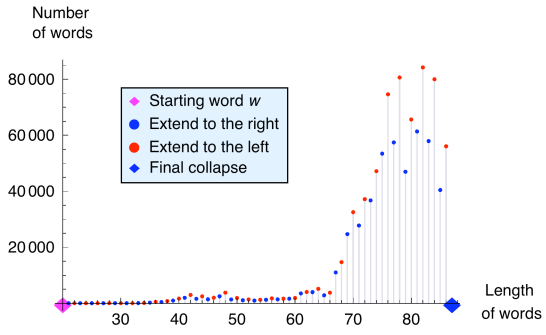

Figure 1. Extend abcbdbcbacbdcdacbdac alternately to the right and left, stepping one letter at a time.

We now provide an explanation for the case shown in the preceding figure. We tried to extend the word w = `"abcbdbcbacbdcdacbdac"` to the right (blue) and to the left (red) with all possible letters over Σ_4. The length of the words increases only by one letter at a time (alternately to the right and left). All possible words are gathered in a list and the number of words is plotted according to their length. For each word there are four possibilities: it can be continued with 3, 2, 1, or 0 letters. The case of 0 letters means that the word cannot be continued (in an a-2-free way) at all. If no words can be continued, then we get an empty list and the computation ends. The resulting list consists merely of unfavorable (or bad, or forbidden) factors that start from the original given word, and this starting word is "in the middle" of every nonempty word in the list. Indeed, all the computations for the words in the list of `someUnfavourableFactors`⋱`OfLength20`, together with a great many other similar words, do end with an empty list (i.e., with a dead end) even though the width of the bidirectional tree, and the computational time, can be huge. All this can make finding unfavorable factors really challenging.

Here are some additional illustrations of this phenomenon. This time, the number of words is depicted using logarithmic scaling.

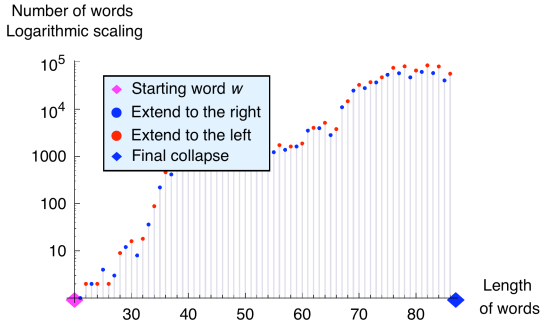

Figure 2. Extend abacadbcbacbdcdacbab alternately to the right and left, stepping one letter at a time.

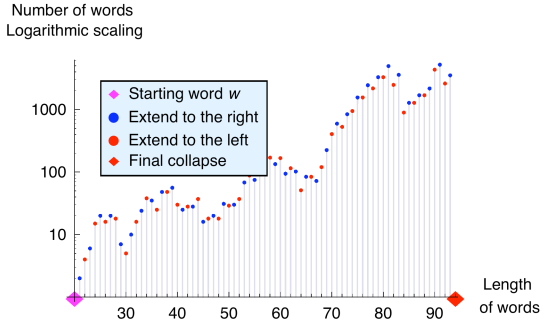

Figure 3. Extend abacadbdadcacbadcbab alternately to the right and left, stepping one letter at a time.

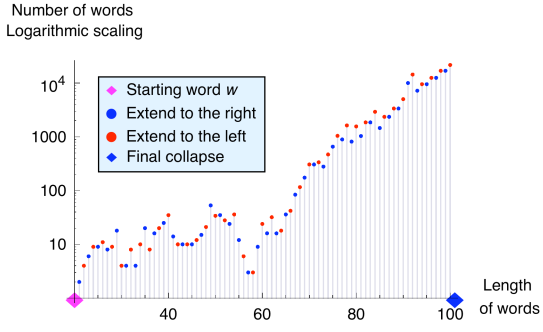

Figure 4. Extend abacdbacbcdcbadacdab alternately to the right and left, stepping one letter at a time.

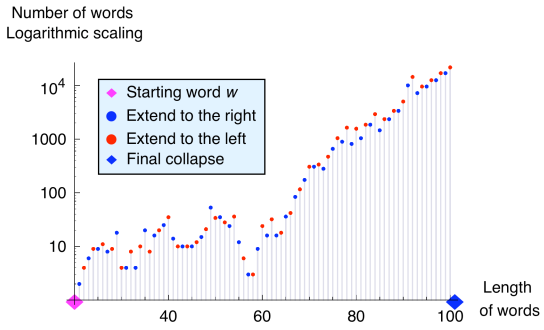

Figure 5. Extend abacdbadbdcdbacadcab alternately to the right and left, stepping one letter at a time.

In the last illustration, the word w = `"abcdacbabdabacdacbcdad"` of length 22 turns out to be unfavorable in a monstrous way. The blue point, just before the final collapse, is at (117, 7866918). In spite of this, not a single extension is possible to the left (which would otherwise lead to words of length 118).

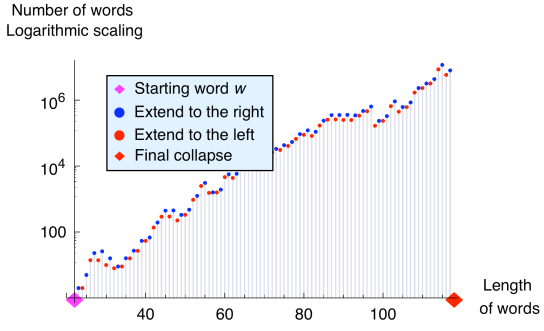

Figure 6. Extend abcdacbabdabacdacbcdad alternately to the right and left, stepping one letter at a time.

In passing, we mention that some unfavorable factors can even be cut shorter from the right or left ends to (shortened) factors which, after a transitional phase, have the same final trajectory.

■ Preliminaries

In this section we present some mathematical notations and terminology. Our terminology is more or less standard in the field of combinatorics on words. Consequently, the reader might consult this section later, if need arises.

An *alphabet* Σ is a finite nonempty set of abstract symbols called *letters*. A *word* (*string*) over Σ is a finite (unless otherwise indicated) string, or sequence, of letters belonging to Σ. The set of all words (nonempty words) over Σ is denoted by Σ^* (Σ^+). On the set Σ^*, the associative binary operation of *catenation* is defined.

For words u and v, it is the juxtaposition uv. The *empty word*, which is the neutral element of catenation, is denoted by λ. The algebraic structures Σ^* and Σ^+ are called, respectively, the free monoid and the free semigroup generated by Σ.

Let $w = x_1 \ldots x_m$, $x_i \in \Sigma$. The *length* of the word w, denoted by $|w|$, is the number of occurrences of letters in w, that is, $|w| = m$. Let $\Sigma = \{a_1, \ldots, a_n\}$. The number of occurrences of one letter $x \in \Sigma$ in w is denoted by $|w|_x$, or simply by $|w|_i$ if $x = a_i$. The notation $\psi_\Sigma(w)$ stands for the *Parikh vector* of w, that is, $\psi_\Sigma(w) = (|w|_1, \ldots, |w|_n)$. Usually we will omit the subscript Σ and write ψ instead of ψ_Σ. Quite interchangeably with the Parikh vector notation, we also use formal sums $\psi_s(w) = \sum_{x \in \{a_1, \ldots, a_n\}} k_x\, x$, with $k_x \in \mathbb{N}$. For example, $\psi_s(abacaba) = 4\,a + 2\,b + c$. Thus $\psi_s(w)$, with w a word over Σ, is an element of the abelian free monoid $\mathbb{N}^\Sigma$ generated by Σ. We will also consider differences of Parikh vectors and differences of formal sums. Consequently, these vectors and sums are extended into elements of the abelian free group $\mathbb{Z}^\Sigma$ generated by Σ. The neutral element of $\mathbb{Z}^\Sigma$ is denoted by $\mathbf{0}$.

A word u is called a *factor* (some authors call it a *subword*) of a word w, if $w = p\,u\,s$ for some words p and s. If $p = \lambda$ ($s = \lambda$), then u is called a *prefix* (a *suffix)* of w.

Let $k \geq 2$ be a given integer. A *k-repetition* (an *abelian k-repetition*) is a nonempty word of the form R^k ($P_1 \ldots P_k$, where $\psi(P_\mu) = \psi(P_\nu)$ for all $1 \leq \mu < \nu \leq k$, that is, the P_i are permutations of each other). Instead of (abelian) 2- and 3-repetitions, the terms (*abelian*) *squares* and (*abelian*) *cubes* are often used. A word or an ω-word (explained later) is called *k-repetition free* (*abelian k-repetition free*, or *k-free in the abelian sense*), or in short *k-free* (*a-k-free*), if it does not contain any k-repetition (abelian k-repetition) as a factor. A word sequence or a word set is *k-free* (*a-k-free*) if all words in it are k-free (a-k-free). If, for a fixed k, it is possible to construct arbitrarily long (infinite) a-k-free (or other pattern-free) words over a given alphabet Σ, then we say that abelian k-repetitions (or those patterns) are *avoidable* over Σ.

A *morphism* h is a mapping between free monoids Σ^* and Δ^* with $h(u\,v) = h(u)\,h(v)$ for every u and v in Σ^*. In particular, $h(\lambda) = \lambda$. A morphism $h : \Sigma^* \to \Delta^*$, being compatible with the catenation of words, is uniquely defined if the word $h(x) \in \Delta^*$ is (effectively) given for each $x \in \Sigma$. If $\Delta = \Sigma$, we call h an *endomorphism* (and usually write g instead of h). For a morphism h and a language L, we define $h(L) = \{h(w) \mid w \in L\}$. A morphism h is termed *uniformly growing* if $|h(x)| = |h(y)| \geq 2$ for every x and $y \in \Sigma$.

For a given integer $k \geq 2$, a morphism $h : \Sigma^* \to \Delta^*$ is called *k-free* (*a-k-free*) if $h(w)$ is k-free (a-k-free) for every k-free (a-k-free) word $w \in \Sigma^*$.

With regards to *L-systems* (Aristid Lindenmayer 1925-1989), we specify the following concepts. A *D0L-system* is a triple $G = (\Sigma, g, \alpha_0)$, where Σ is an alphabet, $g : \Sigma^* \to \Sigma^*$ is an endomorphism, and α_0, called the *axiom*, is a word over Σ. The (*word*) *sequence* $S(G)$ generated by G consists of the words

$$\alpha_0 = g^0(\alpha_0),\ g^1(\alpha_0),\ g^2(\alpha_0), g^3(\alpha_0), \ldots,$$

where $g^i(\alpha_0) = g\left(g^{i-1}(\alpha_0)\right)$ for $i \geq 1$. The *language* of G is defined by $L(G) = \{g^i(\alpha_0) \mid i \geq 0\}$. Languages (sequences) defined by a D0L-system are re-

ferred to as *D0L-languages* (*D0L-sequences*). D0L-systems provide a very convenient way for defining languages and infinite words. Furthermore, if g and α_0 are k-free (a-k-free), then the iteration of g will yield a k-free (a-k-free) D0L-sequence. An *HD0L-system* is a 5-tuple $G_1 = (\Sigma, \Delta, g, h, \alpha_0)$, where (Σ, g, α_0) is a D0L-system, called the underlying D0L-system of G_1, Δ is an alphabet, and $h : \Sigma^* \to \Delta^*$ is a morphism. The *HD0L-sequence* $S(G_1)$ generated by G_1 consists of the words

$$h(\alpha_0) = h\left(g^0(\alpha_0)\right),\ h\left(g^1(\alpha_0)\right),\ h\left(g^2(\alpha_0)\right),\ h\left(g^3(\alpha_0)\right),\ \ldots,$$

and the *HD0L-language* of G_1 is the set $L(G_1) = \left\{h\left(g^i(\alpha_0)\right) \mid i \geq 0\right\}$. A *DT0L-system* is a triple $G_2 = (\Sigma, H, \alpha_0)$, where H is a finite nonempty set of morphisms (called tables) and (Σ, h, α_0) is a D0L-system for every $h \in H$. The *DT0L-language* of G_2 is the set $L(G_2) = \{w \mid w = \alpha_0$ or $w = h_k \ldots h_1(\alpha_0)$, where the compositions $h_k \ldots h_1$ of morphisms are constructed from $h_1, \ldots, h_k \in H\}$. Obviously, a DT0L-system can be regarded as a D0L-system, when H contains only one (endo)morphism. For a thorough discussion of various L-systems the reader is referred to Rozenberg and Salomaa [13].

An *ω-word* is an infinite sequence, from left to right, of letters of an alphabet Σ. Thus an ω-word can be identified with a mapping of $\mathbb{N}_+$ into Σ. One can construct an ω-word, for example, by iterating an endomorphism $g : \Sigma^* \to \Sigma^*$, such that $\lambda \notin g(\Sigma)$ and $g(x) = x\,w$ for some $x \in \Sigma$, $w \in \Sigma^+$. Such a morphism g is called *prefix preserving* for the reason that $g^i(x)$ is a proper prefix of $g^{i+1}(x)$ whenever $i \geq 0$. An ω-word is obtained as the "limit" of the sequence $g^i(a)$; $i = 0, 1, 2, \ldots$.

■ Suppression of Unfavorable Factors with *Mathematica*

In this section we give examples of the developed *Mathematica* program. The correctness of the program and related packages has been tested, but exhaustive computer runs are still likely to take a long time in the future. The full program and further versions of it can be downloaded from [14] or from [15], the latter of which is a general link page for the topic. In addition, the program (in original form) is included at the end of this article, and the reader is invited to experiment with it. For this purpose, one may copy and edit the Input cells of the examples presented. The program consists of initialization cells and will be activated automatically.

In the following examples, many of the function definitions are omitted (they can be found at the end of this article). Note that all of the structures, variables, and constants starting with `€` are global. For example, the global variable `€state` represents the state of the construction. Its values are strings, including, for example, `"extendOrChangeRight"`, `"extendOrChangeLeft"`, `"testRight"`, `"testLeft"`, `"failBoth"`, and `"succeeded"`.

As mentioned in the Introduction, we first fix the alphabet Σ and consider words over it. We take a word (in the final investigation, we will actually take all of the a-2-free words of a given length) and try to extend it in an a-2-free fashion to the right and left in all possible ways up to a given upper bound for the total length.

Each time, the length of the word increases only by a given fixed length. We extend alternately to the right and left, and backtrack when necessary. If the upper bounds are reached, then the original word is so-far-favorable. If there is no way to reach the upper bounds, then the original word is classified, without any doubt, to be unfavorable. At present, favorable words (for the four-letter case) consist of only those occurring as factors in a-2-free words obtained by using known a-2-free endomorphisms and substitutions such as g_{85}, g_{98}, Carpi's [4] modification of g_{85}, and the new examples presented in [7].

In the final program we use integer coding for letters of the alphabet Σ and cumulative integer lists for words. This makes detecting abelian squares fast. We use two different cumulative integer lists, `€cumulIntListRight` for the right-hand extensions and `€cumulIntListLeft` for the left-hand extensions. This makes the addition of integers (addition of cumulative integer lists, in fact) fast, but forces us to write the testing function `testA2RLpairCumulIntList` in a more complicated fashion (nevertheless, still maintaining the necessary high speed). In building all the structures, quite extensive precomputations are needed.

Now we define the alphabets by using letters (strings of length 1) and integers.

```
In[154]:= €alphTwoLet = {"a", "b"};
          €alphTwoInt = {0, 1};
          €alphThreeLet = {"a", "b", "c"};
          €alphThreeInt = {0, 1, 2^16};
          (* Words of length ≤ (2^16-1)*4/3 = 87380
             are safe to use - provided that they do
             not contain xxxx for a letter x in €alphThreeLet *)
          €alphFourLet = {"a", "b", "c", "d"};
          €alphFourInt = {0, 1, 2^10, 2^20};
          (* Words of length ≤ (2^10-1)*2 = 2046
             are safe to use - provided that they do
             not contain xx for a letter x in €alphFourLet *)
          €alphThreeLetInt = {€alphTwoLet, €alphTwoInt};
          €alphThreeLetInt = {€alphThreeLet, €alphThreeInt};
          €alphFourLetInt = {€alphFourLet, €alphFourInt};
```

This first example using cumulative Parikh vectors is symbolic (Parikh vectors are explained in the Preliminaries section).

```
In[163]:= convertStrToCumulIntList["bacabcacab",
           {{"a", "b", "c"}, {"a", "b", "c"}}]

Out[163]= {b, a + b, a + b + c, 2 a + b + c, 2 a + 2 b + c, 2 a + 2 b + 2 c,
           3 a + 2 b + 2 c, 3 a + 2 b + 3 c, 4 a + 2 b + 3 c, 4 a + 3 b + 3 c}
```

In actual computations we use the integer representation for these cumulative Parikh vectors.

```
In[164]:= convertStrToCumulIntList["bacabcacab", €alphThreeLetInt]

Out[164]= {1, 1, 65 537, 65 537, 65 538, 131 074, 131 074, 196 610, 196 610, 196 611}
```

Then we transform back to the string representation over three letters.

```
In[165]:= convertCumulIntListToStr[%, €alphThreeLetInt]
Out[165]= bacabcacab
```

The case of four letters can be handled in a similar way.

```
In[166]:= convertStrToCumulIntList["bacabdacad", €alphFourLetInt]
Out[166]= {1, 1, 1025, 1025, 1026, 1 049 602,
           1 049 602, 1 050 626, 1 050 626, 2 099 202}

In[167]:= convertCumulIntListToStr[%, €alphFourLetInt]
Out[167]= bacabdacad
```

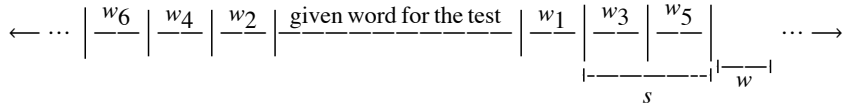

Figure 7. Extend the given word alternately to the right and left. The word *sw* is favorable or so-far-favorable.

We have also taken care that the selection of the next proper candidate is done in an efficient way. If all the possible candidates for w have already been tried out, then the program backtracks to change the other end's suffix. Indeed, the process is the same for the right- and left-hand extensions (both of their cumulative list representations grow from left to right), so we can really speak of suffixes only. The lengths for the suffix s and the extension w need to be fixed at the beginning of the computation, and changing (re-fixing) the lengths usually requires new quite extensive precomputations.

In the program at the end of this article, the lengths have been set as $|s| = 4$ and $|w| = 4$, but they can be, and usually are, selected differently as well. Up to now, we have been using, for example, the lengths $|s| = 8$, $|w| = 4$, and $|s| = 12$, $|w| = 4$. Longer suffixes s would improve the computational time efficiency considerably, but, on the other hand, the structures might need too much memory in our present environment for distributed computing. Moreover, selecting $|w| = 1$ would allow the maximal avoidance of unfavorable factors in the extensions. However, the setting $|w| > 1$ probably allows detecting the abelian squares more quickly, albeit the final decision for this still requires quite extensive experiments.

In the following example, we add all possible words w of length 4, represented by lists that follow all possible reduced suffixes s of length 4. Here the term "reduced" means that we pay attention only to the structure of the word—the other possibilities can straightforwardly be obtained by permutations, that is, by renaming letters. The example originates from the three-letter case in which short abelian repetitions of the form xx or xxx for a letter $x \in \Sigma_3 = \{a, b, c\}$ are allowed. For simplicity, we show the words in their string form.

The next input serves only descriptive purposes and is not intended for actual evaluation inside this notebook.

```
€reducedWordListSuff4Add4 =
 selectWordsWithProperPrefixes[#, addWordList8StartWitha] & /@
  reducedWordList4
```

To save memory and to make the computations as fast as possible, the final calculations in fact use symbols instead of strings. The cumulative integer lists are associated to symbols by using rules. Moreover, to save memory, the permutations are added only when needed. In the following example, we show a rule from symbols to cumulative integer lists (in an abbreviated form).

```
€ruleSymbolsToCumulIntegerLists = {aaab -> {0, 0, 0, 1}, aaba -> {0, 0, 1, 1},
aabb -> {0, 0, 1, 2}, aabc -> {0, 0, 1, 65537}, abaa -> {0, 1, 1, 1}, abac -> {0,
1, 1, 65537}, abbb -> {0, 1, 2, 3}, abbc -> {0, 1, 2, 65538}, abca -> {0, 1,
65537, 65537}, abcb -> {0, 1, 65537, 65538}, abcc -> {0, 1, 65537, 131073},
..., acbb -> {0, 65536, 65537, 65538}, bbba -> {1, 2, 3, 3}, ..., bcaa -> {1,
65537, 65537, 65537}, ccca -> {65536, 131072, 196608, 196608}, ..., cbaa ->
{65536, 65537, 65537, 65537}};
```

In our present setting, we cut the suffix s from the cumulative integer list(s) for the word constructed so far and then select the new extension w from the precomputed symbolic list for s. After this, the corresponding cumulative integer list for w is to be added to (the cumulative integer list of) the previously constructed word. This is done in order to detect the possible new abelian squares quickly. One function we use for selecting the new extensions is `tryProperExtension`⁖`VisList`. Next we present, as an example, one rule (of a great many) connected to it. This time the example is from the four-letter, $|s| = 12$, $|w| = 4$ case.

```
In[168]:= tryProperExtensionVisList[{0, 1, 1, 1025, 1025, 1026,
            1026, 1 049 602, 1 049 603, 1 050 627, 1 050 627, 1 051 651}] =
           {bcda, bcdb, bcdc, daca, dacb, dadb, dcab, dcba, dcbc, dcbd};
```

These kinds of precomputed rules are quite fast to use, but, of course, a considerable amount of memory is needed to store all the structures.

The following function `generateRLthree` (or, equivalently, `generateRL`) constructs the extensions for a given word. Its arguments are states (strings that are also values of the global variable `€state`). As mentioned earlier, some of these states include `"extendOrChangeRight"`, `"extendOrChangeLeft"`, `"testRight"`, `"testLeft"`, `"failBoth"`, and `"succeeded"`. You can see the program for `generateRLthree` by opening the cell brackets at the end of this article. Note that even though the program was originally developed for the three-letter case (allowing short abelian repetitions of the form xx or xxx for a letter $x \in \Sigma_3 = \{a, b, c\}$), it works perfectly for the four-letter case as well in all our settings. Therefore, the more generic name `generateRL` is used for it.

Before starting the construction, we need to initialize the global (at this stage) variables depending on whether we use three or four letters. This is accomplished by using the `initialise` function that has the following main structure.

```
initialise[howManyStepsLeftAndRight_,
 givenWordForTest_, alphLetInt_: €alphThreeLetInt]
```

The variables `€extensionBoundaryLengthRight = howManyStepsLeftAnd`⁖`Right*€addWordLength` and `€extensionBoundaryLengthLeft = howMany`⁖`StepsLeftAndRight*€addWordLength` stand for extension lengths. Both of

these lengths should be equal and of the form k *€addWordLength for a positive integer k. In our examples, and in our present program, the value for €addWordLength (length of w) is equal to 4. Here is the full definition of the initialise function.

```
In[169]:= Clear[initialise];
          initialise[howManyStepsLeftAndRight_,
             givenWordForTest_, alphLetInt_: €alphThreeLetInt] :=
            (€alphLetInt = alphLetInt; (* €alphLetInt needs to be
              set to €alphThreeLetInt or to €alphFourLetInt *)
             If[alphLetInt == €alphThreeLetInt,
              alphThreeLetIntCase, alphFourLetIntCase];
             €addWordLength = 4;
             €preCheckedSuffLength = 8;
             €suffLengthForWhichTryProperExtension =
              €preCheckedSuffLength - €addWordLength;
             €reducedWordListSuffNAddNToExpressions =
              €reducedWordListSuff4Add4ToExpressions;
             ordinalIndexForCumulIntegerListN =
              ordinalIndexForCumulIntegerList4; (* function names *)
             €pointerExtendRight = 0;
          €pointerExtendLeft = 0;
          €extensionBoundaryLengthRight =
              howManyStepsLeftAndRight * €addWordLength;
             €extensionBoundaryLengthLeft =
              howManyStepsLeftAndRight * €addWordLength;
             (* Indeed, both of these should be equal and of the form
              k * €addWordLength for a positive integer k *)
             €howFarExtendedRight = 0;
             €howFarExtendedLeft = 0;
             €indexListRight = {0};
             €indexListLeft = {};
             €givenWordForTest = givenWordForTest;
             €cumulIntListOfGivenWordForTest =
              convertStrToCumulIntList[€givenWordForTest, €alphLetInt];
             €cumulIntListReverseOfGivenWordForTest =
              convertStrToCumulIntList[
               StringReverse[€givenWordForTest], €alphLetInt]);
```

Here is an example of the three-letter case. The program consists of initialization cells and will be activated automatically.

```
In[171]:= initialise[3, "aaabbbaaacccaaabbb"];

          Print["€givenWordForTest = ", €givenWordForTest];
          generateRLthree["startConstruction"];
          While[ €state =!= "failBoth" ⋀ €state =!= "succeeded",
            generateRLthree[€state] ] // Timing
          generateRLthree[€state]
```

```
€givenWordForTest = aaabbbaaacccaaabbb
```

```
Out[174]= {0.018503, Null}
```

```
For aaabbbaaacccaaabbb the extension failed!
```

Thus, the given word is unfavorable.

The next examples deal with the four-letter case. First, we try to extend a word of length 20.

```
In[176]:= "abacabadbacbcdbacdbd" // StringLength
```

```
Out[176]= 20
```

The extra condition in the `While` loop guarantees that the computation will not last too long.

```
In[177]:= initialise[8, "abacabadbacbcdbacdbd", €alphFourLetInt];

          generateRL["startConstruction"]; iii = 1;
          Print["€givenWordForTest = ", €givenWordForTest];
          While[ iii ≤ 50 000 ⋀ €state =!= "failBoth" ⋀ €state =!= "succeeded",
            generateRL[€state]; iii++ ] // Timing
          generateRL[€state]
```

```
€givenWordForTest = abacabadbacbcdbacdbd
```

```
Out[179]= {11.5677, Null}
```

```
For abacabadbacbcdbacdbd the extension was successful!
```

Well, after all, the computation was not too long. At this point, we know only that the given word is so-far-favorable.

Here are some of the inner values and structures produced by the previous computation.

```
In[181]:= myPrint["4 letters"]
```

```
€cumulIntListLeft as reverse
   string | €cumulIntListRight as string =
 cbcacbcdcabdbabcdabcacdadbdadcadabacabadba|
 cbcdbacdbdadbcacdacabdabcbdadcdbcbadbcbdbc
```

```
€howFarExtendedLeft = 0
```

```
€indexListLeft = {23, 1, 2, 12, 4, 24, 12, 25}
```

```
€howFarExtendedRight = 0
```

```
€indexListRight = {24, 11, 3, 21, 20, 12, 14, 34, 0}
```

```
€state = succeeded
```

The lists `€indexListLeft` and `€indexListRight` contain the pointer values that indicate which words w from the precomputed lists should be next catenated for testing to corresponding suffixes s. Variables `€howFarExtendedLeft` and `€howFarExtendedRight` are used for efficient a-2-freeness testing and to find the next proper word w for catenation as efficiently as possible.

Here we test that the extension reached is indeed an a-2-free word.

```
In[182]:= testA2[
           "cbcacbcdcabdbabcdabcacdadbdadcadabacabadbacbcdbacdbdadbcacdaca↘
              bdabcbdadcdbcbadbcbdbc"]
```

```
Out[182]= True
```

We now extend the same word `"abacabadbacbcdbacdbd"` of length 20 one more step (of length 4) to the right and left.

```
In[183]:= initialise[9, "abacabadbacbcdbacdbd", €alphFourLetInt];

          generateRL["startConstruction"]; iii = 1;
          Print["€givenWordForTest = ", €givenWordForTest];
          While[ iii ≤ 100 000 ⋀ €state =!= "failBoth" ⋀ €state =!= "succeeded",
            generateRL[€state]; iii++ ] // Timing
          generateRL[€state]
```

```
€givenWordForTest = abacabadbacbcdbacdbd
```

```
Out[185]= {55.5767, Null}
```

```
Out[186]= testRight
```

Once again, we know only that the given word is so-far-favorable.

Let us consider the following word of length 84.

```
In[187]:= "cbcacbcdcabdbabcdabcacdadbdadcadabacabadbacbcdbacdbdadbcacdacabd\
          abcbdadcdbcbadbcbdbc" // StringLength
Out[187]= 84

In[188]:= initialise[3,
            "cbcacbcdcabdbabcdabcacdadbdadcadabacabadbacbcdbacdbdadbcacdac\
             abdabcbdadcdbcbadbcbdbc", €alphFourLetInt];

          generateRL["startConstruction"]; iii = 1;
          Print["€givenWordForTest = ", €givenWordForTest];
          While[ iii ≤ 100 000 ⋀ €state =!= "failBoth" ⋀ €state =!= "succeeded",
            generateRL[€state]; iii++ ] // Timing
          generateRL[€state]

          €givenWordForTest =
           cbcacbcdcabdbabcdabcacdadbdadcadabacabadbacbcdbacdbdadbcacdaca-
              bdabcbdadcdbcbadbcbdbc

Out[190]= {0.040727, Null}

          For
           cbcacbcdcabdbabcdabcacdadbdadcadabacabadbacbcdbacdbdadbcacdaca-
              bdabcbdadcdbcbadbcbdbc the extension failed!
```

In this case we know definitely that the given word of length 84 is unfavorable. Actually, it turns out that the given word is already the longest possible extension of w = "abacabadbacbcdbacdbd"! Note that the €givenWordForTest has the form of "cbcacbcdcabdbabcdabcacdadbdadcad" <> w <> "adbcacdacabdabcbdadcdbcbadbcbdbc". However, we will not elaborate on this here.

Here is another word to consider.

```
In[192]:= initialise[8, "abcbdbcbacbdcdacbdac", €alphFourLetInt];

          generateRL["startConstruction"];
          Print["€givenWordForTest = ", €givenWordForTest];
          While[€state =!= "failBoth" ⋀ €state =!= "succeeded",
            generateRL[€state]; ] // Timing
          generateRL[€state]

          €givenWordForTest = abcbdbcbacbdcdacbdac

Out[194]= {15.1336, Null}

          For abcbdbcbacbdcdacbdac the extension was successful!
```

Thus, for the time being, our case is so-far-favorable. The computation did not take long even though the extension was quite large ($8 \times 4 = 32$ to both sides).

Here are some of the inner values and structures produced by the previous computation.

```
In[196]:= myPrint["4 letters"]
```

```
€cumulIntListLeft as reverse
   string | €cumulIntListRight as string =
 abdabacbdbadbdcdadbabcacdbcabadbabcbdbcbac|
 bdcdacbdacabacdbdabdbcabcdcabadacdcbacdcac
```

```
€howFarExtendedLeft = 0
```

```
€indexListLeft = {10, 12, 14, 18, 19, 10, 16, 13}
```

```
€howFarExtendedRight = 0
```

```
€indexListRight = {28, 40, 13, 32, 25, 23, 16, 23, 0}
```

```
€state = succeeded
```

We now test that the extension reached really is an a-2-free word.

```
In[197]:= testA2[
           "abdabacbdbadbdcdadbabcacdbcabadbabcbdbcbacbdcdacbdacabacdbdabd↖
              bcabcdcabadacdcbacdcac"]
```

```
Out[197]= True
```

This is correct!

The final example shows that our word "abcbdbcbacbdcdacbdac" turns out to be unfavorable. However, this time the computation takes much more time than in the previous example.

```
In[198]:= initialise[9, "abcbdbcbacbdcdacbdac", €alphFourLetInt];

          generateRL["startConstruction"];
          Print["€givenWordForTest = ", €givenWordForTest];
          While[€state =!= "failBoth" ∧ €state =!= "succeeded",
            generateRL[€state]; ] // Timing
          generateRL[€state]
```

```
€givenWordForTest = abcbdbcbacbdcdacbdac
```

```
Out[200]= {8023.36, Null}
```

```
For abcbdbcbacbdcdacbdac the extension failed!
```

We might have expected that the long buffers (found before this final trial) of length $8 \times 4 = 32$ in both directions for "abcbdbcbacbdcdacbdac" would guarantee the given word to be favorable. Surprisingly enough, this turns out not to be the case.

■ Conclusions

Mathematica has versatile structures that firmly support the development of concepts, which has been extremely useful for constructing prototypes and full standalone programs. The use of *Mathematica* has enabled us to discover phenomena that previously were either unbelievable or very hard to experiment with. Even so, it would be useful, if in the future, in *Mathematica*, one could also use restricted pattern matching in a fast way as explained in the Introduction. Lastly, we expect that the presented program and its future updates will be used for a long time in our research.

■ Acknowledgments

We gratefully acknowledge the participation of the following individuals, most of whom have been students at the Rovaniemi Polytechnic/University of Applied Sciences over the course of the research from 1990 to 2006, who have written a number of computer programs to search for strings with desirable properties, helped in a crucial way to set up the computing environments, or made interactive graphical and musical representations of the structures. Starting year is given in parentheses: Kari Tuovinen (1990); Minna Iivonen, Anja Keskinarkaus, Marko Manninen (1993); Abdeljalil Chabani, Tomi Laakso (1994); Mika Moilanen, Juha Särestöniemi (1996); Juho Alfthan (1999); Olli-Pentti Saira (2000); Marja Kenttä, Ville Mattila (2001); Lauri Autio, Marianna Mölläri (2002); Antti Eskola (2003); Antti Karhu, Veli-Matti Lahtela, Olli-Pekka Siivola (2004); Esa Nyrhinen, Sami Vuolli (2005); Esa Taskila, Mikhail Kalkov (2006).

■ References

[1] P. Erdös, "Some Unsolved Problems," *Magyar Tudomanyos Akademia Matematikai Kutato Intezet Kozlemenye,* **6**, 1961 pp. 221-254.

[2] P. A. B. Pleasants, "Non-Repetitive Sequences," *Proceedings of the Cambridge Philosophical Society*, **68**, 1970 pp. 267-274.

[3] V. Keränen, "Abelian Squares Are Avoidable on 4 Letters," in *Proceedings of the Nineteenth International Colloquium on Automata, Languages and Programming (ICALP '92)*, Vienna (W. Kuich, ed.), *Lecture Notes in Computer Science*, **623**, London: Springer-Verlag, 1992 pp. 41-52.

[4] A. Carpi, "On the Number of Abelian Square-Free Words on Four Letters," *Discrete Applied Mathematics*, **81**(1-3), 1998 pp. 155-167. doi:10.1016/S0166-218X (97)88002-X.

[5] V. Keränen, "*Mathematica* in Word Pattern Avoidance Research," CADE—2007: *Computer Algebra and Differential Equations* (A. Mylläri, V. Edneral, and N. Ourusoff , eds.), *Acta Academiae Aboensis, Series B, Mathematica et physica*, **67**(2), Åbo: Åbo Akademi University Press, 2007 pp. 12-27. urn.fi/URN:ISBN:978-951-765-403-6.

[6] V. Keränen. "A-2-Free Endomorphisms and Substitutions over 4 Letters." (Mar 29, 2007) www.south.rotol.ramk.fi/keranen/words2007/a2f.html.

[7] V. Keränen, "A Powerful Abelian Square-Free Substitution over 4 Letters," *Theoretical Computer Science*, 2009. doi:10.1016/j.tcs.2009.05.027.

[8] R. L. Rivest. "Abelian Square-Free Dithering for Iterated Hash Functions." Draft, Cambridge, MA: MIT (2005) www.theory.lcs.mit.edu/~rivest/publications.html.

[9] E. Andreeva, C. Bouillaguet, P-A. Fouque, J. J. Hoch, J. Kelsey, A. Shamir, and S. Zimmer, "Second Preimage Attacks on Dithered Hash Functions," in *Advances in Cryptology, Proceedings of the Twenty-Seventh Annual International Conference on the Theory and Applications of Cryptographic Techniques (EUROCRYPT'08)*, Istanbul (N. P. Smart, ed.), *Lecture Notes in Computer Science*, **4965**, Berlin: Springer, 2008 pp. 270-288.

[10] S. Mäkelä, "Patterns in Words," M.Sc. Thesis. University of Turku, Turku, Finland, 2002 (in Finnish).

[11] P. Ochem and T. Reix. "Upper Bound on the Number of Ternary Square-Free Words." (2006) www.lri.fr/~ochem/morphisms/wowa.ps.

[12] V. Keränen. "Forbidden Abelian Square-Free Factors over 4 Letters." (Jun 8, 2004) www.south.rotol.ramk.fi/keranen/research/ForbiddenFactors.html.

[13] G. Rozenberg and A. Salomaa, *The Mathematical Theory of L-systems (Pure and Applied Mathematics)*, New York: Academic Press, 1980.

[14] V. Keränen. "Programs & Notebooks & Packages (*Mathematica* Programs for Suppression of Unfavourable Factors in Pattern Avoidance)." (Mar 22, 2006) www.south.rotol.ramk.fi/keranen/research/UnfavourableFactorsInPatternAvoidance/Programs&Notebooks&Packages.html.

[15] V. Keränen. "Avoidable Regularities in Strings: Structures, Programs, Graphics, and Music of A-2-Free Strings,1996-2009." (2009) www.south.rotol.ramk.fi/keranen/StructuresGraphicsMusic.html.

Veikko Keränen, "Combinatorics on Words," *The Mathematica Journal*, 2010. dx.doi.org/doi:10.3888/tmj.11.3–4.

About the Author

Veikko Keränen has been Principal Lecturer in Mathematics at the Rovaniemi University of Applied Sciences since 1987. He received his Ph.D. in mathematics from the University of Oulu in 1986. Keränen is one of the founding members, with Peter Mitic and Gautam Dasgupta, of the International *Mathematica* Symposium (IMS). His research areas include theoretical computer science, combinatorics on words, avoidable regularities in strings, and symbolic computation. Keränen has also been involved in mathematics teaching development, for example, through the Matriculation Examination Board and the National Board of Education of Finland.

Veikko Keränen
Rovaniemi University of Applied Sciences
Jokiväylä 11
96300 Rovaniemi
Finland
veikko.keranen@ramk.fi
veikko.keranen@gmail.com

■ Programs for Three and Four Letters

Parts of the programs below are in a kind of preliminary form, though, most likely, they will be sufficient for all experimental purposes.

```
(*************************************************************
  ****
  This file was generated automatically by the Mathematica
 front end. It contains Initialization cells from a Notebook file,
which typically will have the same name as this
 file except ending in ".nb" instead of ".m".
*************************************************************
 **)

Off[General::spell];
Off[General::spell1];

(* SetDirectory["D:\_IMS2006"]; *)
(*Write here your own directory for reading and saving files*)

(* all structures,
variables and constants starting with € are global *)
€alphTwoLet = {"a", "b"};
€alphTwoInt = {0, 1};
€alphThreeLet = {"a", "b", "c"};
€alphThreeInt = {0, 1, 2^16};
(* Words of length ≤ (2^16-1)*4/3 = 87380
    are safe to use - provided that they do
    not contain xxxx for a letter x in €alphThreeLet *)
€alphFourLet = {"a", "b", "c", "d"};
€alphFourInt = {0, 1, 2^10, 2^20};
(* Words of length ≤ (2^10-1)*2 = 2046
    are safe to use - provided that they
    do not contain xx for a letter x in €alphFourLet *)
€alphThreeLetInt = {€alphTwoLet, €alphTwoInt};
€alphThreeLetInt = {€alphThreeLet, €alphThreeInt};
€alphFourLetInt = {€alphFourLet, €alphFourInt};

Clear[convertStrToCumulIntList,
  convertCumulIntListToStr, convertSuffNCumulIntListToStr,
  testA2Ffor1LetterLeft, testA2Ffor1LetterRight];

convertStrToCumulIntList[x_String, {alphLet_List, alphInt_List}] :=
```

```
  Drop[FoldList[Plus, 0, ReplaceAll[Characters[x],
     Thread[Rule[alphLet, alphInt]]]], 1];

convertCumulIntListToStr[x_List,
   {alphLet_List, alphInt_List}] := ReplaceAll[
    Map[#[[2]] - #[[1]] &, Partition[Flatten[{0, x}], 2, 1]],
    Thread[Rule[alphInt, alphLet]]] // StringJoin;

convertSuffNCumulIntListToStr[
   x_List, {alphLet_List, alphInt_List}, n_] :=
  (ReplaceAll[Map[#[[2]] - #[[1]] &,
       Partition[Take[x, -n - 1], 2, 1]], Thread[
       Rule[alphInt, alphLet]]] // StringJoin) /; n < Length[x];

convertSuffNCumulIntListToStr[
   x_List, {alphLet_List, alphInt_List}, n_] :=
  convertCumulIntListToStr[x, {alphLet, alphInt}] /;
   n ≥ Length[x];

(*
convertStrToCumulIntList[x_String,4]:=
 Drop[FoldList[Plus,0,ReplaceAll[Characters[x],
    {"a"→0,"b"→1,"c"→2^10,"d"→2^20}]],1] (* alphSize=4 *);

convertCumulIntListToStr[x_List,4]:=
 ReplaceAll[Map[#[[2]]-#[[1]]&,Partition[Flatten[{0,x}],2,1]],
   {0→"a",1→"b",2^10→"c",2^20→"d"}]//StringJoin; *)

Clear[selectSuffixes];
selectSuffixes[pref_String, word_List] :=
  {pref, StringCases[#, StartOfString ~~ pref ~~ x___ ~~ EndOfString →
        x] & /@ word // Flatten};

Clear[cutPref];
cutPref[{x_String, y_String}] := StringCases[
    x <> "." <> y, (p___ ~~ s1__ ~~ "." ~~ p___ ~~ s2__ → s2)][[1]];

Clear[selectSuffixes];
selectSuffixes[pref_String, word_List] :=
  {pref, StringCases[#, StartOfString ~~ pref ~~ x___ ~~ EndOfString →
        x] & /@ word // Flatten};
(*Save["reducedAddWordList8.txt",reducedAddWordList8];
Save["extendWordList4by4.txt",extendWordList4by4];*)

Clear[cutPref];
cutPref[{x_String, y_String}] := StringCases[
    x <> "." <> y, (p___ ~~ s1__ ~~ "." ~~ p___ ~~ s2__ → s2)][[1]];
```

```
Clear[formPrefSuffPairs];
formPrefSuffPairs[{x_String, y_String}] :=
  StringCases[x, {p__ ~~ y → {p, y}, y → {x, ""}}][[1]];

Clear[selectSuffixes];
selectSuffixes[pref_String, word_List] := {pref,
  StringCases[#, StartOfString ~~ pref ~~ x___ ~~ EndOfString → x] & /@
    word // Flatten}

howManyToAppendToRight :=
  Length[€reducedWordListSuffNAddNToExpressions[[
    ordinalIndexForCumulIntegerListN[Take[€cumulIntListRight,
      -€suffLengthForWhichTryProperExtension]], 2]]];
howManyToAppendToLeft := Length[
   €reducedWordListSuffNAddNToExpressions[[
    ordinalIndexForCumulIntegerListN[Take[€cumulIntListLeft,
      -€suffLengthForWhichTryProperExtension]], 2]]];

Clear[whichToCatenateRL]; (*cumulIntList must be of length≥
 €suffLengthForWhichTryProperExtension+1*)
whichToCatenateRL[cumulIntList_, pointerExtendRL_] :=
 tryProperExtension[
  Take[cumulIntList, -€suffLengthForWhichTryProperExtension] -
   cumulIntList[[Length[cumulIntList] -
     €suffLengthForWhichTryProperExtension]], pointerExtendRL];
Clear[whichToCatenateWithKnownSuffRL];
whichToCatenateWithKnownSuffRL[suff_, pointerExtendRL_] :=
  tryProperExtension[suff, pointerExtendRL];

Clear[catenateRL]; (*cumulIntList must be of length≥
 €suffLengthForWhichTryProperExtension+1*)
catenateRL[cumulIntList_, pointerExtendRL_] :=
 Join[cumulIntList, Last[cumulIntList] +
   whichToCatenateRL[cumulIntList, pointerExtendRL]];
Clear[catenateWithKnownAddRL];
catenateWithKnownAddRL[cumulIntList_, whichToCatenateRL_] :=
  Join[cumulIntList, Last[cumulIntList] + whichToCatenateRL];

(*Clear[nextProperTrialForNewExtension];
nextProperTrialForNewExtension[suff_,
  previousCatenationRL_,howFarExtendedRight_,pointerExtendRL_]:=
 Module[{localExtendRL=pointerExtendRL,
   prefToCheck=Take[previousCatenationRL,howFarExtendedRight+1],
   whatToCatenate,prefOfwhatToCatenate,
   jjPointerForTest,successTF=True},
  While[If[localExtendRL≤Length[tryProperExtension[suff]],True,
```

```
      successTF=False]&&(prefToCheck===(prefOfwhatToCatenate=
         Take[whatToCatenate=whichToCatenateWithKnownSuffRL[
            suff,localExtendRL],howFarExtendedRight+1])),
    localExtendRL++];€pointerExtendRight=localExtendRL;
  If[successTF,jjPointerForTest=
    Position[prefOfwhatToCatenate-prefToCheck,_?(#≠0&)][[1,1]],
    jjPointerForTest=0];{successTF,whatToCatenate,
    jjPointerForTest}];*)

Clear[nextProperTrialForNewExtension2nd];
(*here the pointerExtendRL refers to present place-not the next*)
nextProperTrialForNewExtension2nd[suff_, 0, 0] :=
 {True, whichToCatenateWithKnownSuffRL[suff, 1], 1, 1};
nextProperTrialForNewExtension2nd[suff_, pointerExtendRL_, _] :=
  {False, {}, 0, 0} /;
   pointerExtendRL ≥ Length[tryProperExtension[suff]];

nextProperTrialForNewExtension2nd[suff_, pointerExtendRL_, 0] :=
  {True, whichToCatenateWithKnownSuffRL[suff, pointerExtendRL + 1],
    pointerExtendRL + 1, 1} /; pointerExtendRL > 0;

nextProperTrialForNewExtension2nd[
   suff_, pointerExtendRL_, howFarExtendedRight_] :=
  Module[{localExtendRL = pointerExtendRL + 1,
    prefToCheck = Take[whichToCatenateWithKnownSuffRL[
       suff, If[pointerExtendRL == 0, 1, pointerExtendRL]],
      howFarExtendedRight + 1], whatToCatenate,
    prefOfwhatToCatenate, jjPointerForTest, successTF = True},
   While[If[localExtendRL ≤ Length[tryProperExtension[suff]],
      True, successTF = False] &&
     (prefToCheck === (prefOfwhatToCatenate = Take[whatToCatenate =
           whichToCatenateWithKnownSuffRL[suff, localExtendRL],
         howFarExtendedRight + 1])), localExtendRL ++];
   If[successTF, jjPointerForTest = Position[
       prefOfwhatToCatenate - prefToCheck, _?(# ≠ 0 &)][[1, 1]],
    jjPointerForTest = 0]; {successTF, whatToCatenate,
    localExtendRL, jjPointerForTest}];

Clear[nextProperTrialForNewExtensionFin];
nextProperTrialForNewExtensionFin[
  cumulIntList_, pointerExtendRL_, howFarExtendedRight_] :=
 nextProperTrialForNewExtension2nd[
  Take[cumulIntList, -€suffLengthForWhichTryProperExtension] -
   cumulIntList[[Length[cumulIntList] -
     €suffLengthForWhichTryProperExtension]],
  pointerExtendRL, howFarExtendedRight];
```

```
Clear[testA2RLpairCumulIntList];
(*testA2RLpairCumulIntList works also in the case of €alphFourLet,
if preCheckedSuffLength-addedOrReplacedWordLength>
 1. Below x refers to €cumulIntListRight and y refers
  to €cumulIntListLeft,or the other way round.*)

testA2RLpairCumulIntList[x_List, y_List,
   preCheckedSuffLength_, addedOrReplacedWordLength_, jj_: 1] :=
  Module[{rLen = Length[x] - addedOrReplacedWordLength,
    lLen = Length[y], j = jj, res = True}, While[
    j ≤ addedOrReplacedWordLength, i = Floor[(preCheckedSuffLength -
          addedOrReplacedWordLength + j)/2] + 1;
    While[(i < Floor[(rLen + j)/2]) &&
      (res = (x[[rLen + j]] - x[[rLen + j - i]] ===
           x[[rLen + j - i]] - x[[rLen + j - 2 i]] // Not)), i++];
    If[(2 i ≤ rLen + j) && res, If[OddQ[rLen + j],
      res = (x[[rLen + j]] - x[[Ceiling[(rLen + j)/2]]] ===
          x[[Ceiling[(rLen + j)/2]]] - x[[1]] // Not);
      i++, res = (x[[rLen + j]] - x[[(rLen + j)/2]] ===
          x[[(rLen + j)/2]] // Not); i++]];
    If[res // Not, Return[{res, j}]];
    While[(i < rLen + j) && (2 i ≤ lLen + rLen + j) &&
      (res = (x[[rLen + j]] - x[[rLen + j - i]] ===
           x[[rLen + j - i]] + y[[2 i - rLen - j]] // Not)), i++];
    If[res, If[(2 i ≤ lLen + rLen + j), res =
       (x[[rLen + j]] === y[[2 i - rLen - j]] // Not)]];
    If[res // Not, Return[{res, j}]];
    i++;
    While[
     (2 i ≤ lLen + rLen + j) && (res = (x[[rLen + j]] + y[[i - rLen - j]] ===
           y[[2 i - rLen - j]] - y[[i - rLen - j]] // Not)), i++];
    If[res // Not, Return[{res, j}]];
    j++];
   {res, j - 1}];

(*€bottomBoundaryLengthRight and €bottomBoundaryLengthLeft will
  be computed in generateRLthree["startConstruction"].The
  maximum length (total) is €bottomBoundaryLengthRight+
 €extensionBoundaryLengthRight+€bottomBoundaryLengthLeft+
 €extensionBoundaryLengthLeft*)

(*for example;
€preCheckedSuffLength-€addWordLength and €preCheckedSuffLength-
 €addedOrReplacedWordLength
```

```
   (which is kind of dynamically changing variable) represent the
   length of the a2free factor that does not require testing*)

Clear[generateRLthree];
generateRLthree["startConstruction"] :=
   (€wordLeft = StringTake[€givenWordForTest,
      Ceiling[StringLength[€givenWordForTest]/2]];
    €wordRight = StringTake[€givenWordForTest,
      -Floor[StringLength[€givenWordForTest]/2]];
    €cumulIntListLeft = convertStrToCumulIntList[
      StringReverse[€wordLeft], €alphLetInt];
    (*€cumulIntListLeft also grows from left to right*)
    €cumulIntListRight =
     convertStrToCumulIntList[€wordRight, €alphLetInt];
    €bottomBoundaryLengthRight = Length[€cumulIntListRight];
    €bottomBoundaryLengthLeft = Length[€cumulIntListLeft];
    €state = "extendRightBottomEnd";);

generateRLthree["extendOrChangeRight"] :=
   If[Length[€cumulIntListLeft] ≥ €bottomBoundaryLengthLeft +
       €extensionBoundaryLengthLeft, €howFarExtendedRight = 0;
     €state = "succeeded", €howFarExtendedRight = 0;
     €indexListLeft = Append[€indexListLeft, 0];
     €state = "extendOrChangeLeft"] /; Length[€cumulIntListRight] ≥
     €bottomBoundaryLengthRight + €extensionBoundaryLengthRight;

generateRLthree["extendOrChangeRight"] :=
   If[({{€successTF, €whatToCatenateRight,
        €pointerExtendRight, €howFarExtendedRight}} =
       nextProperTrialForNewExtensionFin[€cumulIntListRight,
        Last[€indexListRight], €howFarExtendedRight])[[1]],
    €cumulIntListRight = catenateWithKnownAddRL[
      €cumulIntListRight, €whatToCatenateRight];
    €indexListRight = ReplacePart[€indexListRight,
      €pointerExtendRight, Length[€indexListRight]];
    €state = "testRight", €indexListRight = Take[€indexListRight,
      Length[€indexListRight] - 1]; €state = "changeLeft"];

generateRLthree["testRight"] :=
   Module[{testResult = testA2RLpairCumulIntList[€cumulIntListRight,
       €cumulIntListLeft, €preCheckedSuffLength,
       €addWordLength, €howFarExtendedRight]},
    If[testResult[[1]], €howFarExtendedRight = 0;
     €indexListLeft = Append[€indexListLeft, 0];
```

```
    €state = "extendOrChangeLeft",
    €howFarExtendedRight = testResult[[2]] - 1;
    €cumulIntListRight = Take[€cumulIntListRight,
      Length[€cumulIntListRight] - €addWordLength];
    €state = "extendOrChangeRight"]];

generateRLthree["extendOrChangeLeft"] :=
  If[Length[€cumulIntListRight] ≥ €bottomBoundaryLengthRight +
      €extensionBoundaryLengthRight, €howFarExtendedLeft = 0;
    €state = "succeeded", €howFarExtendedLeft = 0;
    €indexListRight = Append[€indexListRight, 0];
    €state = "extendOrChangeRight"] /; Length[€cumulIntListLeft] ≥
    €bottomBoundaryLengthLeft + €extensionBoundaryLengthLeft;

generateRLthree["extendOrChangeLeft"] :=
  If[({€successTF, €whatToCatenateLeft,
       €pointerExtendLeft, €howFarExtendedLeft} =
      nextProperTrialForNewExtensionFin[€cumulIntListLeft,
       Last[€indexListLeft], €howFarExtendedLeft])[[1]],
   €cumulIntListLeft = catenateWithKnownAddRL[
     €cumulIntListLeft, €whatToCatenateLeft];
   €indexListLeft = ReplacePart[€indexListLeft,
     €pointerExtendLeft, Length[€indexListLeft]];
   €state = "testLeft", €indexListLeft = Take[€indexListLeft,
     Length[€indexListLeft] - 1]; €state = "changeRight"];

generateRLthree["testLeft"] :=
  Module[{testResult = testA2RLpairCumulIntList[€cumulIntListLeft,
      €cumulIntListRight, €preCheckedSuffLength,
      €addWordLength, €howFarExtendedLeft]},
   If[testResult[[1]], €howFarExtendedLeft = 0;
    €indexListRight = Append[€indexListRight, 0];
    €state = "extendOrChangeRight",
    €howFarExtendedLeft = testResult[[2]] - 1;
    €cumulIntListLeft = Take[€cumulIntListLeft,
      Length[€cumulIntListLeft] - €addWordLength];
    €state = "extendOrChangeLeft"]];

generateRLthree["changeRight"] := (€state = "failBoth") /;
   ((Length[€cumulIntListRight] == €bottomBoundaryLengthRight) ∧
     (Length[€cumulIntListLeft] == €bottomBoundaryLengthLeft));
generateRLthree["changeRight"] := (€state = "changeLeft") /;
   ((Length[€cumulIntListRight] == €bottomBoundaryLengthRight) ∧
     (Length[€cumulIntListLeft] > €bottomBoundaryLengthLeft));
```

```
generateRLthree["changeRight"] :=
  (€cumulIntListRight = Take[€cumulIntListRight,
      Length[€cumulIntListRight] - €addWordLength];
    €howFarExtendedRight = 0; €state = "extendRightBottomEnd") /;
   Length[€cumulIntListRight] ==
    €bottomBoundaryLengthRight + €addWordLength;

generateRLthree["changeRight"] :=
  (€cumulIntListRight = Take[€cumulIntListRight,
      Length[€cumulIntListRight] - €addWordLength];
    €howFarExtendedRight = 0; €state = "extendOrChangeRight") /;
   Length[€cumulIntListRight] >
    €bottomBoundaryLengthRight + €addWordLength;

generateRLthree["extendRightBottomEnd"] :=
  If[({€successTF, €whatToCatenateRight, €pointerExtendRight,
       €howFarExtendedRight} = nextProperTrialForNewExtensionFin[
       €cumulIntListOfGivenWordForTest,
       Last[€indexListRight], €howFarExtendedRight])[[1]],
   €cumulIntListRight = catenateWithKnownAddRL[
     €cumulIntListRight, €whatToCatenateRight];
   €indexListRight = ReplacePart[€indexListRight,
     €pointerExtendRight, Length[€indexListRight]];
   €state = "testRightBottomEnd", €indexListRight =
    Take[€indexListRight, Length[€indexListRight] - 1];
   €state = "changeLeft"];

generateRLthree["testRightBottomEnd"] :=
  Module[{testResult = testA2RLpairCumulIntList[€cumulIntListRight,
      €cumulIntListLeft, €preCheckedSuffLength,
      €addWordLength, €howFarExtendedRight]},
   If[testResult[[1]], €howFarExtendedRight = 0;
    €indexListLeft = Append[€indexListLeft, 0];
    €state = "extendLeftBottomEnd",
    €howFarExtendedRight = testResult[[2]] - 1;
    €cumulIntListRight = Take[€cumulIntListRight,
      Length[€cumulIntListRight] - €addWordLength];
    €state = "extendRightBottomEnd"]];

generateRLthree["changeLeft"] := (€state = "failBoth") /;
   ((Length[€cumulIntListLeft] == €bottomBoundaryLengthLeft) ∧
     (Length[€cumulIntListRight] == €bottomBoundaryLengthRight));
generateRLthree["changeLeft"] := (€state = "changeRight") /;
   ((Length[€cumulIntListLeft] == €bottomBoundaryLengthLeft) ∧
```

```
      (Length[€cumulIntListRight] > €bottomBoundaryLengthRight));

generateRLthree["changeLeft"] :=
    (€cumulIntListLeft = Take[€cumulIntListLeft,
        Length[€cumulIntListLeft] - €addWordLength];
      €howFarExtendedLeft = 0; €state = "extendLeftBottomEnd") /;
     Length[€cumulIntListLeft] ==
      €bottomBoundaryLengthLeft + €addWordLength;;

generateRLthree["changeLeft"] :=
  (€cumulIntListLeft = Take[€cumulIntListLeft,
      Length[€cumulIntListLeft] - €addWordLength];
    €howFarExtendedLeft = 0; €state = "extendOrChangeLeft") /;
   Length[€cumulIntListLeft] >
    €bottomBoundaryLengthLeft + €addWordLength;

generateRLthree["extendLeftBottomEnd"] :=
  If[({€successTF, €whatToCatenateLeft, €pointerExtendLeft,
       €howFarExtendedLeft} = nextProperTrialForNewExtensionFin[
       €cumulIntListReverseOfGivenWordForTest,
       Last[€indexListLeft], €howFarExtendedLeft])[[1]],
   €cumulIntListLeft = catenateWithKnownAddRL[
     €cumulIntListLeft, €whatToCatenateLeft];
   €indexListLeft = ReplacePart[€indexListLeft,
     €pointerExtendLeft, Length[€indexListLeft]];
   €state = "testLeftBottomEnd", €indexListLeft =
    Take[€indexListLeft, Length[€indexListLeft] - 1];
   €state = "changeRight"];

generateRLthree["testLeftBottomEnd"] :=
  Module[{testResult = testA2RLpairCumulIntList[€cumulIntListLeft,
      €cumulIntListRight, €preCheckedSuffLength,
      €addWordLength, €howFarExtendedLeft]},
   If[testResult[[1]], €howFarExtendedLeft = 0;
    €indexListRight = Append[€indexListRight, 0];
    €state = "extendOrChangeRight",
    €howFarExtendedLeft = testResult[[2]] - 1;
    €cumulIntListLeft = Take[€cumulIntListLeft,
      Length[€cumulIntListLeft] - €addWordLength];
    €state = "extendLeftBottomEnd"]];

generateRLthree["failBoth"] :=
  Print["For ", €givenWordForTest, " the extension failed!"];
generateRLthree["succeeded"] := Print["For ",
   €givenWordForTest, " the extension was successful!"];
```

```
generateRL = generateRLthree ;(* Indeed,
also the four letter case,in which no abelian squares are allowed,
will work all right in our setting! *)

Clear[initialise];
initialise[howManyStepsLeftAndRight_,
    givenWordForTest_, alphLetInt_: €alphThreeLetInt] :=
  (€alphLetInt = alphLetInt; (* €alphLetInt needs to be
     set to €alphThreeLetInt or to €alphFourLetInt *)
    If[alphLetInt == €alphThreeLetInt,
     alphThreeLetIntCase, alphFourLetIntCase];
    (* Orignally this read as follows:
       If[alphLetInt==€alphThreeLetInt,
        <<"alphThreeLetIntCase.m",<<"alphFourLetIntCase.m"]; *)
    €addWordLength = 4;
    €preCheckedSuffLength = 8;
    €suffLengthForWhichTryProperExtension =
     €preCheckedSuffLength - €addWordLength;
    €reducedWordListSuffNAddNToExpressions =
     €reducedWordListSuff4Add4ToExpressions;
    ordinalIndexForCumulIntegerListN =
     ordinalIndexForCumulIntegerList4; (* function names *)
    €pointerExtendRight = 0;
€pointerExtendLeft = 0;
€extensionBoundaryLengthRight =
     howManyStepsLeftAndRight * €addWordLength;
    €extensionBoundaryLengthLeft =
     howManyStepsLeftAndRight * €addWordLength;
    (* Indeed, both of these should be equal and of the form
     k * €addWordLength for a positive integer k *)
    €howFarExtendedRight = 0;
    €howFarExtendedLeft = 0;
    €indexListRight = {0};
    €indexListLeft = {};
    €givenWordForTest = givenWordForTest;
    €cumulIntListOfGivenWordForTest =
     convertStrToCumulIntList[€givenWordForTest, €alphLetInt];
    €cumulIntListReverseOfGivenWordForTest =
     convertStrToCumulIntList[
      StringReverse[€givenWordForTest], €alphLetInt]);


Clear[s];
s[x_List] := Apply[Plus, x];
s[x_String] := Apply[Plus, Characters[x]];

Clear[test3Suff];
```

```
test3Suff[x_] := Module[
   {sl = StringLength[x], i = 1, res = True}, While[i < Floor[sl/2] &&
     (res = s[StringTake[x, {sl - 1 - 2 i, sl - 1 - i}]] ===
          s[StringTake[x, {sl - i, sl}]] // Not), i++];
   res];

Clear[test3A2];
test3A2[x_, a2FreePrefLen_: 4] :=
  If[StringLength[x] < 4, "Too short string",
   Module[{sl = StringLength[x], j = 0, res = True},
    While[a2FreePrefLen + j ≤ sl &&
      (res = test3Suff[StringTake[x, a2FreePrefLen + j]]), j++];
    (asp = a2FreePrefLen + j; res)]];

Clear[test3SuffVis];
test3SuffVis[x_] :=
  Module[{sl = StringLength[x], i = 1, res}, While[i < Floor[sl/2] &&
     (res = s[StringTake[x, {sl - 1 - 2 i, sl - 1 - i}]] ===
          s[StringTake[x, {sl - i, sl}]] // Not), i++];
   If[res, True, {StringTake[x, {sl - 1 - 2 i, sl - 1 - i}],
     StringTake[x, {sl - i, sl}], 2 (i + 1)}]];

Clear[test3A2Vis];
test3A2Vis[x_, a2FreePrefLen_: 4] :=
  If[StringLength[x] < 2, "Too short string", Module[
    {sl = StringLength[x], i = 0, res}, While[a2FreePrefLen + i ≤ sl,
     If[(res = test3SuffVis[StringTake[x, a2FreePrefLen + i]]) ===
       True, True, Print[res]]; i++];
    "End"]];

Clear[testSuff];
testSuff[x_] := Module[
   {sl = StringLength[x], i = 0, res = True}, While[i < Floor[sl/2] &&
     (res = s[StringTake[x, {sl - 1 - 2 i, sl - 1 - i}]] ===
          s[StringTake[x, {sl - i, sl}]] // Not), i++];
   res];

Clear[testSuffVis];
testSuffVis[x_] :=
  Module[{sl = StringLength[x], i = 0, res}, While[i < Floor[sl/2] &&
     (res = s[StringTake[x, {sl - 1 - 2 i, sl - 1 - i}]] ===
          s[StringTake[x, {sl - i, sl}]] // Not), i++];
   If[res, True, {StringTake[x, {sl - 1 - 2 i, sl - 1 - i}],
     StringTake[x, {sl - i, sl}], 2 (i + 1)}]];
```

```
Clear[testA2];
testA2[x_, a2FreePrefLen_: 2] :=
  If[StringLength[x] < 2, "Too short string", Module[
    {sl = StringLength[x], i = 0, res}, While[a2FreePrefLen + i ≤ sl &&
      (res = testSuff[StringTake[x, a2FreePrefLen + i]]), i++];
    (asp = a2FreePrefLen + i; res)]];

Clear[testA2Mir];
testA2Mir[x_, a2FreePrefLen_: 2] :=
  testA2[StringReverse[x], a2FreePrefLen];

Clear[testA2Vis];
testA2Vis[x_, a2FreePrefLen_: 2] :=
 If[StringLength[x] < 2, "Too short string", Module[
   {sl = StringLength[x], i = 0, res}, While[a2FreePrefLen + i ≤ sl,
    If[(res = testSuffVis[StringTake[x, a2FreePrefLen + i]]) ===
      True, True, Print[res]]; i++];
   "End"]]

Clear[testA2MirVis];
testA2MirVis[x_, a2FreePrefLen_: 2] :=
  testA2Vis[StringReverse[x], a2FreePrefLen];

Clear[myPrint];
myPrint["3 letters"] :=
  (Print["€cumulIntListLeft as reverse string | €cumulIntListRight
      as string = ", convertCumulIntListToStr[
      €cumulIntListLeft, €alphThreeLetInt] // StringReverse,
    "|", convertCumulIntListToStr[€cumulIntListRight,
     €alphThreeLetInt]];
   Print["€howFarExtendedLeft = ", €howFarExtendedLeft];
   Print["€indexListLeft = ", €indexListLeft];
   Print["€howFarExtendedRight = ", €howFarExtendedRight];
   Print["€indexListRight = ", €indexListRight];
   Print["€state = ", €state]);

myPrint["4 letters"] :=
 (Print["€cumulIntListLeft as reverse string | €cumulIntListRight
     as string = ", convertCumulIntListToStr[
     €cumulIntListLeft, €alphFourLetInt] // StringReverse,
   "|", convertCumulIntListToStr[€cumulIntListRight,
    €alphFourLetInt]];
  Print["€howFarExtendedLeft = ", €howFarExtendedLeft];
  Print["€indexListLeft = ", €indexListLeft];
  Print["€howFarExtendedRight = ", €howFarExtendedRight];
  Print["€indexListRight = ", €indexListRight];
```

```
  Print["€state = ", €state])

Clear[s];
s[x_List] := Apply[Plus, x];
s[x_String] := Apply[Plus, Characters[x]];
Clear[testSuff];
testSuff[x_] := Module[
  {sl = StringLength[x], i = 0, res = True}, While[i < Floor[sl/2] &&
    (res = s[StringTake[x, {sl - 1 - 2 i, sl - 1 - i}]] ===
        s[StringTake[x, {sl - i, sl}]] // Not), i++];
  res]

Clear[testSuffVis];
testSuffVis[x_] :=
 Module[{sl = StringLength[x], i = 0, res}, While[i < Floor[sl/2] &&
    (res = s[StringTake[x, {sl - 1 - 2 i, sl - 1 - i}]] ===
        s[StringTake[x, {sl - i, sl}]] // Not), i++];
  If[res, True, {StringTake[x, {sl - 1 - 2 i, sl - 1 - i}],
    StringTake[x, {sl - i, sl}], 2 (i + 1)}]]

Clear[testA2];
testA2[x_, a2FreePrefLen_: 2] :=
 If[StringLength[x] < 2, "Too short string", Module[
   {sl = StringLength[x], i = 0, res}, While[a2FreePrefLen + i ≤ sl &&
     (res = testSuff[StringTake[x, a2FreePrefLen + i]]), i++];
   (asp = a2FreePrefLen + i; res)]]


Clear[testA2Mir];
testA2Mir[x_, a2FreePrefLen_: 2] :=
 testA2[StringReverse[x], a2FreePrefLen]

Clear[testA2Vis];
testA2Vis[x_, a2FreePrefLen_: 2] :=
 If[StringLength[x] < 2, "Too short string", Module[
   {sl = StringLength[x], i = 0, res}, While[a2FreePrefLen + i ≤ sl,
    If[(res = testSuffVis[StringTake[x, a2FreePrefLen + i]]) ===
      True, True, Print[res]]; i++];
   "End"]]

Clear[testA2MirVis];
testA2MirVis[x_, a2FreePrefLen_: 2] :=
 testA2Vis[StringReverse[x], a2FreePrefLen]

Clear[testAllPattern];
testAllPattern[
```

```
   {___, x_, _, x_, _, x_, _, x_, _, x_, _, x_, ___}] := False;
testAllPattern[{___, a_, b_, c_, a_, d_, b_, a_, d_, ___}] := False;
testAllPattern[{___, d_, a_, b_, d_, a_, c_, b_, a_, ___}] := False;
testAllPattern[{___, a_, b_, a_, c_, d_, a_, b_, a_, ___}] := False;
testAllPattern[{___, a_, b_, a_, c_, d_, b_, a_, b_, ___}] := False;
testAllPattern[{___, a_, b_, c_, a_, b_, d_, a_, b_, ___}] := False;
testAllPattern[_] := True;

Clear[allPerm];
allPerm[x_String] :=
 StringReplace[x, Thread[Rule[{"a", "b", "c", "d"}, #]]] & /@
  Permutations[{"a", "b", "c", "d"}]

Clear[deletePermutations];
deletePermutations[{x_List, y_List}] :=
 {Append[x, y[[1]]], Complement[Rest[y], allPerm[y[[1]]]]}

Clear[deleteMirrorPermutations];
deleteMirrorPermutations[{x_List, y_List}] := {Append[x, y[[1]]],
  Complement[Rest[y], allPerm[StringReverse[y[[1]]]]]}

Clear[prod, myRepl1, myRepl2, myRepl];
prod = {"a" → "a_,", "b" → "b_,", "c" → "c_,", "d" → "d_,"};
myRepl1[x_String] := StringReplace[x, prod];
myRepl2[x_String] := "{___," <> x <> "___}";
myRepl[x_String] := myRepl2[myRepl1[x]];

Clear[abacXabcaXabcbXabcdPREF];
abacXabcaXabcbXabcdPREF[{a_, b_, a_, c_}] := {a → "a", b → "b",
   c → "c", Complement[{"a", "b", "c", "d"}, {a, b, c}][[1]] → "d"};
abacXabcaXabcbXabcdPREF[{a_, b_, c_, a_}] := {a → "a", b → "b",
   c → "c", Complement[{"a", "b", "c", "d"}, {a, b, c}][[1]] → "d"};
abacXabcaXabcbXabcdPREF[{a_, b_, c_, b_}] := {a → "a", b → "b",
   c → "c", Complement[{"a", "b", "c", "d"}, {a, b, c}][[1]] → "d"};
abacXabcaXabcbXabcdPREF[{a_, b_, c_, d_}] :=
  {a → "a", b → "b", c → "c", d → "d"} /;
   Sort[{a, b, c, d}] === {"a", "b", "c", "d"};

Clear[abacXabcaXabcbXabcdRule];
(*same as abacXabcaXabcbXabcdPREF*)
abacXabcaXabcbXabcdRule[{a_, b_, a_, c_}] := {a → "a", b → "b",
  c → "c", Complement[{"a", "b", "c", "d"}, {a, b, c}][[1]] → "d"};
abacXabcaXabcbXabcdRule[{a_, b_, c_, a_}] := {a → "a", b → "b",
   c → "c", Complement[{"a", "b", "c", "d"}, {a, b, c}][[1]] → "d"};
abacXabcaXabcbXabcdRule[{a_, b_, c_, b_}] := {a → "a", b → "b",
   c → "c", Complement[{"a", "b", "c", "d"}, {a, b, c}][[1]] → "d"};
abacXabcaXabcbXabcdRule[{a_, b_, c_, d_}] :=
  {a → "a", b → "b", c → "c", d → "d"} /;
   Sort[{a, b, c, d}] === {"a", "b", "c", "d"};
```

```
Clear[abacXabcaXabcbXabcdFORM];
abacXabcaXabcbXabcdFORM[x_String] := StringReplace[x,
   abacXabcaXabcbXabcdPREF[Characters[StringTake[x, 4]]]];

Clear[abacXabcaXabcbXabcdFactorFORM];
abacXabcaXabcbXabcdFactorFORM[x_String, factorPointer_] :=
 StringReplace[x, abacXabcaXabcbXabcdRule[
   Characters[StringTake[x, {factorPointer, factorPointer + 3}]]]];
Clear[myR1Select, myL1Select, myRL];
myR1Select[x_String] := If[testSuff[x], x, {}];
myL1Select[x_String] := If[testSuff[StringReverse[x]], x, {}];
myRL[{w_List, len_, state_}] :=
  Module[{wList = ((myR1Select[#] & /@ catenate1Right[#]) & /@ w) //
       Flatten}, {wList, Length[wList], "L"}] /; state == "R";
myRL[{w_List, len_, state_}] := Module[{wList =
       ((myL1Select[#] & /@ catenate1Left[#]) & /@ w) // Flatten},
     {wList, Length[wList], "R"}] /; state == "L";

words2 = {"ab", "ac", "ad", "ba",
   "bc", "bd", "ca", "cb", "cd", "da", "db", "dc"};

words3 = {"aba", "abc", "abd", "aca", "acb",
   "acd", "ada", "adb", "adc", "bab", "bac", "bad", "bca",
   "bcb", "bcd", "bda", "bdb", "bdc", "cab", "cac", "cad",
   "cba", "cbc", "cbd", "cda", "cdb", "cdc", "dab", "dac",
   "dad", "dba", "dbc", "dbd", "dca", "dcb", "dcd"};

words4 = {"abac", "abad", "abca", "abcb", "abcd", "abda", "abdb",
   "abdc", "acab", "acad", "acba", "acbc", "acbd", "acda", "acdb",
   "acdc", "adab", "adac", "adba", "adbc", "adbd", "adca", "adcb",
   "adcd", "babc", "babd", "baca", "bacb", "bacd", "bada", "badb",
   "badc", "bcab", "bcac", "bcad", "bcba", "bcbd", "bcda", "bcdb",
   "bcdc", "bdab", "bdac", "bdad", "bdba", "bdbc", "bdca", "bdcb",
   "bdcd", "caba", "cabc", "cabd", "cacb", "cacd", "cada",
   "cadb", "cadc", "cbab", "cbac", "cbad", "cbca", "cbcd",
   "cbda", "cbdb", "cbdc", "cdab", "cdac", "cdad", "cdba",
   "cdbc", "cdbd", "cdca", "cdcb", "daba", "dabc", "dabd",
   "daca", "dacb", "dacd", "dadb", "dadc", "dbab", "dbac",
   "dbad", "dbca", "dbcb", "dbcd", "dbda", "dbdc", "dcab",
   "dcac", "dcad", "dcba", "dcbc", "dcbd", "dcda", "dcdb"};
```

The program in the package `alphFourLetIntCase.m`

```
(*************************************************************
  ****
  This file was generated automatically by the Mathematica
 front end. It contains Initialization cells from a Notebook file,
```

```
which typically will have the same name as this file
 except ending in ".nb" instead of ".m". This file is
 intended to be loaded into the Mathematica kernel using
 the package loading commands Get or Needs. Doing so is
 equivalent to using the Evaluate Initialization Cells
 menu command in the front end. DO NOT EDIT THIS FILE. This
 entire file is regenerated automatically each time the
 parent Notebook file is saved in the Mathematica front
 end. Any changes you make to this file will be overwritten.
***********************************************************
 *******)
alphFourLetIntCase := (Off[General::spell];
Off[General::spell1];

  Clear[cutPref];
cutPref[{x_String, y_String}] := StringCases[x <> "." <> y,
      (p___ ~~ s1__ ~~ "." ~~ p___ ~~ s2__ → s2)][[1]];

  Clear[formPrefSuffPairs];
formPrefSuffPairs[{x_String, y_String}] :=
    StringCases[x, {p__ ~~ y → {p, y}, y → {x, ""}}][[1]];

  Clear[selectSuffixes];
selectSuffixes[pref_String, word_List] :=
   {pref, StringCases[#, StartOfString ~~ pref ~~
           x___ ~~ EndOfString → x] & /@ word // Flatten};

  €reducedWordListSuff4Add4ToExpressions =
   {{abac, {abad, abda, abdb, abdc, adab, adac, adba, adbc, adbd,
      adca, adcb, adcd, babd, bada, badb, badc, bcda, bcdb, bcdc,
      bdab, bdac, bdad, bdba, bdbc, bdca, bdcb, bdcd, daba,
      dabc, dabd, daca, dacb, dadb, dbab, dbac, dbad, dbca,
      dbcb, dbcd, dbda, dbdc, dcab, dcac, dcba, dcbc, dcbd}},
    {abca, {bada, badb, badc, bdab, bdac, bdad, bdba, bdbc,
      bdca, bdcb, bdcd, cdab, cdac, cdad, cdba, cdbc, cdbd, cdca,
      cdcb, daba, dabc, dabd, daca, dacb, dbab, dbac, dbad, dbca,
      dbcb, dbcd, dbda, dbdc, dcab, dcac, dcba, dcbc, dcbd, dcdb}},
    {abcb, {abcd, abda, abdb, abdc, adab, adac, adba, adbc,
      adbd, adca, adcb, adcd, daba, dabc, dabd, daca,
      dacb, dacd, dadb, dadc, dbab, dbac, dbad, dbca,
      dbcb, dcab, dcac, dcad, dcba, dcbc, dcda}},
    {abcd, {abac, abad, abca, abcb, abda, abdb, acab, acba,
      acbc, acdc, adba, adbd, adcd, babc, babd, baca, bacb,
      bada, badb, bcab, bcac, bcba, bdab, bdad, caba, cabc,
      cacb, cacd, cada, cadc, cbab, cbac, cbca, cbcd}}};

  €ruleSymbolsToCumulIntegerLists =
   {abac → {0, 1, 1, 1025}, abca → {0, 1, 1025, 1025},
    abcb → {0, 1, 1025, 1026}, abcd → {0, 1, 1025, 1 049 601},
```

```
abad → {0, 1, 1, 1 048 577}, abda → {0, 1, 1 048 577, 1 048 577},
abdb → {0, 1, 1 048 577, 1 048 578},
abdc → {0, 1, 1 048 577, 1 049 601}, acab → {0, 1024, 1024, 1025},
acba → {0, 1024, 1025, 1025}, acbc → {0, 1024, 1025, 2049},
acbd → {0, 1024, 1025, 1 049 601}, acad →
 {0, 1024, 1024, 1 049 600}, acda → {0, 1024, 1 049 600, 1 049 600},
acdc → {0, 1024, 1 049 600, 1 050 624},
acdb → {0, 1024, 1 049 600, 1 049 601},
adab → {0, 1 048 576, 1 048 576, 1 048 577},
adba → {0, 1 048 576, 1 048 577, 1 048 577},
adbd → {0, 1 048 576, 1 048 577, 2 097 153},
adbc → {0, 1 048 576, 1 048 577, 1 049 601},
adac → {0, 1 048 576, 1 048 576, 1 049 600},
adca → {0, 1 048 576, 1 049 600, 1 049 600},
adcd → {0, 1 048 576, 1 049 600, 2 098 176},
adcb → {0, 1 048 576, 1 049 600, 1 049 601},
babc → {1, 1, 2, 1026}, bacb → {1, 1, 1025, 1026},
baca → {1, 1, 1025, 1025}, bacd → {1, 1, 1025, 1 049 601},
babd → {1, 1, 2, 1 048 578}, badb → {1, 1, 1 048 577, 1 048 578},
bada → {1, 1, 1 048 577, 1 048 577},
badc → {1, 1, 1 048 577, 1 049 601}, bcba → {1, 1025, 1026, 1026},
bcab → {1, 1025, 1025, 1026}, bcac → {1, 1025, 1025, 2049},
bcad → {1, 1025, 1025, 1 049 601}, bcbd →
 {1, 1025, 1026, 1 049 602}, bcdb → {1, 1025, 1 049 601, 1 049 602},
bcdc → {1, 1025, 1 049 601, 1 050 625},
bcda → {1, 1025, 1 049 601, 1 049 601},
bdba → {1, 1 048 577, 1 048 578, 1 048 578},
bdab → {1, 1 048 577, 1 048 577, 1 048 578},
bdad → {1, 1 048 577, 1 048 577, 2 097 153},
bdac → {1, 1 048 577, 1 048 577, 1 049 601},
bdbc → {1, 1 048 577, 1 048 578, 1 049 602},
bdcb → {1, 1 048 577, 1 049 601, 1 049 602},
bdcd → {1, 1 048 577, 1 049 601, 2 098 177},
bdca → {1, 1 048 577, 1 049 601, 1 049 601},
cacb → {1024, 1024, 2048, 2049},
cabc → {1024, 1024, 1025, 2049}, caba → {1024, 1024, 1025, 1025},
cabd → {1024, 1024, 1025, 1 049 601},
cacd → {1024, 1024, 2048, 1 050 624},
cadc → {1024, 1024, 1 049 600, 1 050 624},
cada → {1024, 1024, 1 049 600, 1 049 600},
cadb → {1024, 1024, 1 049 600, 1 049 601},
cbca → {1024, 1025, 2049, 2049},
cbac → {1024, 1025, 1025, 2049}, cbab → {1024, 1025, 1025, 1026},
cbad → {1024, 1025, 1025, 1 049 601},
cbcd → {1024, 1025, 2049, 1 050 625},
cbdc → {1024, 1025, 1 049 601, 1 050 625},
cbdb → {1024, 1025, 1 049 601, 1 049 602},
cbda → {1024, 1025, 1 049 601, 1 049 601},
cdca → {1024, 1 049 600, 1 050 624, 1 050 624},
cdac → {1024, 1 049 600, 1 049 600, 1 050 624},
cdad → {1024, 1 049 600, 1 049 600, 2 098 176},
```

```
  cdab → {1024, 1 049 600, 1 049 600, 1 049 601},
  cdcb → {1024, 1 049 600, 1 050 624, 1 050 625},
  cdbc → {1024, 1 049 600, 1 049 601, 1 050 625},
  cdbd → {1024, 1 049 600, 1 049 601, 2 098 177},
  cdba → {1024, 1 049 600, 1 049 601, 1 049 601},
  dadb → {1 048 576, 1 048 576, 2 097 152, 2 097 153},
  dabd → {1 048 576, 1 048 576, 1 048 577, 2 097 153},
  daba → {1 048 576, 1 048 576, 1 048 577, 1 048 577},
  dabc → {1 048 576, 1 048 576, 1 048 577, 1 049 601},
  dadc → {1 048 576, 1 048 576, 2 097 152, 2 098 176},
  dacd → {1 048 576, 1 048 576, 1 049 600, 2 098 176},
  daca → {1 048 576, 1 048 576, 1 049 600, 1 049 600},
  dacb → {1 048 576, 1 048 576, 1 049 600, 1 049 601},
  dbda → {1 048 576, 1 048 577, 2 097 153, 2 097 153},
  dbad → {1 048 576, 1 048 577, 1 048 577, 2 097 153},
  dbab → {1 048 576, 1 048 577, 1 048 577, 1 048 578},
  dbac → {1 048 576, 1 048 577, 1 048 577, 1 049 601},
  dbdc → {1 048 576, 1 048 577, 2 097 153, 2 098 177},
  dbcd → {1 048 576, 1 048 577, 1 049 601, 2 098 177},
  dbcb → {1 048 576, 1 048 577, 1 049 601, 1 049 602},
  dbca → {1 048 576, 1 048 577, 1 049 601, 1 049 601},
  dcda → {1 048 576, 1 049 600, 2 098 176, 2 098 176},
  dcad → {1 048 576, 1 049 600, 1 049 600, 2 098 176},
  dcac → {1 048 576, 1 049 600, 1 049 600, 1 050 624},
  dcab → {1 048 576, 1 049 600, 1 049 600, 1 049 601},
  dcdb → {1 048 576, 1 049 600, 2 098 176, 2 098 177},
  dcbd → {1 048 576, 1 049 600, 1 049 601, 2 098 177},
  dcbc → {1 048 576, 1 049 600, 1 049 601, 1 050 625},
  dcba → {1 048 576, 1 049 600, 1 049 601, 1 049 601}};

listOfRules = {{"a" → "a", "b" → "b", "c" → "c", "d" → "d"},
  {"a" → "a", "b" → "b", "c" → "d", "d" → "c"},
  {"a" → "a", "b" → "c", "c" → "b", "d" → "d"},
  {"a" → "a", "b" → "c", "c" → "d", "d" → "b"},
  {"a" → "a", "b" → "d", "c" → "b", "d" → "c"},
  {"a" → "a", "b" → "d", "c" → "c", "d" → "b"},
  {"a" → "b", "b" → "a", "c" → "c", "d" → "d"},
  {"a" → "b", "b" → "a", "c" → "d", "d" → "c"},
  {"a" → "b", "b" → "c", "c" → "a", "d" → "d"},
  {"a" → "b", "b" → "c", "c" → "d", "d" → "a"},
  {"a" → "b", "b" → "d", "c" → "a", "d" → "c"},
  {"a" → "b", "b" → "d", "c" → "c", "d" → "a"},
  {"a" → "c", "b" → "a", "c" → "b", "d" → "d"},
  {"a" → "c", "b" → "a", "c" → "d", "d" → "b"},
  {"a" → "c", "b" → "b", "c" → "a", "d" → "d"},
  {"a" → "c", "b" → "b", "c" → "d", "d" → "a"},
  {"a" → "c", "b" → "d", "c" → "a", "d" → "b"},
  {"a" → "c", "b" → "d", "c" → "b", "d" → "a"},
  {"a" → "d", "b" → "a", "c" → "b", "d" → "c"},
  {"a" → "d", "b" → "a", "c" → "c", "d" → "b"},
  {"a" → "d", "b" → "b", "c" → "a", "d" → "c"},
```

```
     {"a" → "d", "b" → "b", "c" → "c", "d" → "a"},
     {"a" → "d", "b" → "c", "c" → "a", "d" → "b"},
     {"a" → "d", "b" → "c", "c" → "b", "d" → "a"}};

allWords4ForRules =
   {{"abac", "abca", "abcb", "abcd", "abad", "abda", "abdb", "abdc",
     "acab", "acba", "acbc", "acbd", "acad", "acda", "acdc", "acdb",
     "adab", "adba", "adbd", "adbc", "adac", "adca", "adcd", "adcb",
     "babc", "bacb", "baca", "bacd", "babd", "badb", "bada", "badc",
     "bcba", "bcab", "bcac", "bcad", "bcbd", "bcdb", "bcdc", "bcda",
     "bdba", "bdab", "bdad", "bdac", "bdbc", "bdcb", "bdcd",
     "bdca", "cacb", "cabc", "caba", "cabd", "cacd", "cadc",
     "cada", "cadb", "cbca", "cbac", "cbab", "cbad", "cbcd",
     "cbdc", "cbdb", "cbda", "cdca", "cdac", "cdad", "cdab",
     "cdcb", "cdbc", "cdbd", "cdba", "dadb", "dabd", "daba",
     "dabc", "dadc", "dacd", "daca", "dacb", "dbda", "dbad",
     "dbab", "dbac", "dbdc", "dbcd", "dbcb", "dbca", "dcda",
     "dcad", "dcac", "dcab", "dcdb", "dcbd", "dcbc", "dcba"},
    {"abad", "abda", "abdb", "abdc", "abac", "abca", "abcb",
     "abcd", "adab", "adba", "adbd", "adbc", "adac", "adca", "adcd",
     "adcb", "acab", "acba", "acbc", "acbd", "acad", "acda", "acdc",
     "acdb", "babd", "badb", "bada", "badc", "babc", "bacb", "baca",
     "bacd", "bdba", "bdab", "bdad", "bdac", "bdbc", "bdcb", "bdcd",
     "bdca", "bcba", "bcab", "bcac", "bcad", "bcbd", "bcdb", "bcdc",
     "bcda", "dadb", "dabd", "daba", "dabc", "dadc", "dacd",
     "daca", "dacb", "dbda", "dbad", "dbab", "dbac", "dbdc",
     "dbcd", "dbcb", "dbca", "dcda", "dcad", "dcac", "dcab",
     "dcdb", "dcbd", "dcbc", "dcba", "cacb", "cabc", "caba",
     "cabd", "cacd", "cadc", "cada", "cadb", "cbca", "cbac",
     "cbab", "cbad", "cbcd", "cbdc", "cbdb", "cbda", "cdca",
     "cdac", "cdad", "cdab", "cdcb", "cdbc", "cdbd", "cdba"},
    {"acab", "acba", "acbc", "acbd", "acad", "acda", "acdc",
     "acdb", "abac", "abca", "abcb", "abcd", "abad", "abda", "abdb",
     "abdc", "adac", "adca", "adcd", "adcb", "adab", "adba", "adbd",
     "adbc", "cacb", "cabc", "caba", "cabd", "cacd", "cadc", "cada",
     "cadb", "cbca", "cbac", "cbab", "cbad", "cbcd", "cbdc", "cbdb",
     "cbda", "cdca", "cdac", "cdad", "cdab", "cdcb", "cdbc", "cdbd",
     "cdba", "babc", "bacb", "baca", "bacd", "babd", "badb",
     "bada", "badc", "bcba", "bcab", "bcac", "bcad", "bcbd",
     "bcdb", "bcdc", "bcda", "bdba", "bdab", "bdad", "bdac",
     "bdbc", "bdcb", "bdcd", "bdca", "dadc", "dacd", "daca",
     "dacb", "dadb", "dabd", "daba", "dabc", "dcda", "dcad",
     "dcac", "dcab", "dcdb", "dcbd", "dcbc", "dcba", "dbda",
     "dbad", "dbab", "dbac", "dbdc", "dbcd", "dbcb", "dbca"},
    {"acad", "acda", "acdc", "acdb", "acab", "acba", "acbc",
     "acbd", "adac", "adca", "adcd", "adcb", "adab", "adba", "adbd",
     "adbc", "abac", "abca", "abcb", "abcd", "abad", "abda", "abdb",
     "abdc", "cacd", "cadc", "cada", "cadb", "cacb", "cabc", "caba",
     "cabd", "cdca", "cdac", "cdad", "cdab", "cdcb", "cdbc", "cdbd",
     "cdba", "cbca", "cbac", "cbab", "cbad", "cbcd", "cbdc", "cbdb",
     "cbda", "dadc", "dacd", "daca", "dacb", "dadb", "dabd",
```

```
  "daba", "dabc", "dcda", "dcad", "dcac", "dcab", "dcdb",
  "dcbd", "dcbc", "dcba", "dbda", "dbad", "dbab", "dbac",
  "dbdc", "dbcd", "dbcb", "dbca", "babc", "bacb", "baca",
  "bacd", "babd", "badb", "bada", "badc", "bcba", "bcab",
  "bcac", "bcad", "bcbd", "bcdb", "bcdc", "bcda", "bdba",
  "bdab", "bdad", "bdac", "bdbc", "bdcb", "bdcd", "bdca"},
 {"adab", "adba", "adbd", "adbc", "adac", "adca", "adcd",
  "adcb", "abad", "abda", "abdb", "abdc", "abac", "abca", "abcb",
  "abcd", "acad", "acda", "acdc", "acdb", "acab", "acba", "acbc",
  "acbd", "dadb", "dabd", "daba", "dabc", "dadc", "dacd", "daca",
  "dacb", "dbda", "dbad", "dbab", "dbac", "dbdc", "dbcd", "dbcb",
  "dbca", "dcda", "dcad", "dcac", "dcab", "dcdb", "dcbd", "dcbc",
  "dcba", "babd", "badb", "bada", "badc", "babc", "bacb",
  "baca", "bacd", "bdba", "bdab", "bdad", "bdac", "bdbc",
  "bdcb", "bdcd", "bdca", "bcba", "bcab", "bcac", "bcad",
  "bcbd", "bcdb", "bcdc", "bcda", "cacd", "cadc", "cada",
  "cadb", "cacb", "cabc", "caba", "cabd", "cdca", "cdac",
  "cdad", "cdab", "cdcb", "cdbc", "cdbd", "cdba", "cbca",
  "cbac", "cbab", "cbad", "cbcd", "cbdc", "cbdb", "cbda"},
 {"adac", "adca", "adcd", "adcb", "adab", "adba", "adbd",
  "adbc", "acad", "acda", "acdc", "acdb", "acab", "acba", "acbc",
  "acbd", "abad", "abda", "abdb", "abdc", "abac", "abca", "abcb",
  "abcd", "dadc", "dacd", "daca", "dacb", "dadb", "dabd", "daba",
  "dabc", "dcda", "dcad", "dcac", "dcab", "dcdb", "dcbd", "dcbc",
  "dcba", "dbda", "dbad", "dbab", "dbac", "dbdc", "dbcd", "dbcb",
  "dbca", "cacd", "cadc", "cada", "cadb", "cacb", "cabc",
  "caba", "cabd", "cdca", "cdac", "cdad", "cdab", "cdcb",
  "cdbc", "cdbd", "cdba", "cbca", "cbac", "cbab", "cbad",
  "cbcd", "cbdc", "cbdb", "cbda", "babd", "badb", "bada",
  "badc", "babc", "bacb", "baca", "bacd", "bdba", "bdab",
  "bdad", "bdac", "bdbc", "bdcb", "bdcd", "bdca", "bcba",
  "bcab", "bcac", "bcad", "bcbd", "bcdb", "bcdc", "bcda"},
 {"babc", "bacb", "baca", "bacd", "babd", "badb", "bada",
  "badc", "bcba", "bcab", "bcac", "bcad", "bcbd", "bcdb", "bcdc",
  "bcda", "bdba", "bdab", "bdad", "bdac", "bdbc", "bdcb", "bdcd",
  "bdca", "abac", "abca", "abcb", "abcd", "abad", "abda", "abdb",
  "abdc", "acab", "acba", "acbc", "acbd", "acad", "acda", "acdc",
  "acdb", "adab", "adba", "adbd", "adbc", "adac", "adca", "adcd",
  "adcb", "cbca", "cbac", "cbab", "cbad", "cbcd", "cbdc",
  "cbdb", "cbda", "cacb", "cabc", "caba", "cabd", "cacd",
  "cadc", "cada", "cadb", "cdcb", "cdbc", "cdbd", "cdba",
  "cdca", "cdac", "cdad", "cdab", "dbda", "dbad", "dbab",
  "dbac", "dbdc", "dbcd", "dbcb", "dbca", "dadb", "dabd",
  "daba", "dabc", "dadc", "dacd", "daca", "dacb", "dcdb",
  "dcbd", "dcbc", "dcba", "dcda", "dcad", "dcac", "dcab"},
 {"babd", "badb", "bada", "badc", "babc", "bacb", "baca",
  "bacd", "bdba", "bdab", "bdad", "bdac", "bdbc", "bdcb", "bdcd",
  "bdca", "bcba", "bcab", "bcac", "bcad", "bcbd", "bcdb", "bcdc",
  "bcda", "abad", "abda", "abdb", "abdc", "abac", "abca", "abcb",
  "abcd", "adab", "adba", "adbd", "adbc", "adac", "adca", "adcd",
  "adcb", "acab", "acba", "acbc", "acbd", "acad", "acda", "acdc",
```

```
  "acdb", "dbda", "dbad", "dbab", "dbac", "dbdc", "dbcd",
  "dbcb", "dbca", "dadb", "dabd", "daba", "dabc", "dadc",
  "dacd", "daca", "dacb", "dcdb", "dcbd", "dcbc", "dcba",
  "dcda", "dcad", "dcac", "dcab", "cbca", "cbac", "cbab",
  "cbad", "cbcd", "cbdc", "cbdb", "cbda", "cacb", "cabc",
  "caba", "cabd", "cacd", "cadc", "cada", "cadb", "cdcb",
  "cdbc", "cdbd", "cdba", "cdca", "cdac", "cdad", "cdab"},
 {"bcba", "bcab", "bcac", "bcad", "bcbd", "bcdb", "bcdc",
  "bcda", "babc", "bacb", "baca", "bacd", "babd", "badb", "bada",
  "badc", "bdbc", "bdcb", "bdcd", "bdca", "bdba", "bdab", "bdad",
  "bdac", "cbca", "cbac", "cbab", "cbad", "cbcd", "cbdc", "cbdb",
  "cbda", "cacb", "cabc", "caba", "cabd", "cacd", "cadc", "cada",
  "cadb", "cdcb", "cdbc", "cdbd", "cdba", "cdca", "cdac", "cdad",
  "cdab", "abac", "abca", "abcb", "abcd", "abad", "abda",
  "abdb", "abdc", "acab", "acba", "acbc", "acbd", "acad",
  "acda", "acdc", "acdb", "adab", "adba", "adbd", "adbc",
  "adac", "adca", "adcd", "adcb", "dbdc", "dbcd", "dbcb",
  "dbca", "dbda", "dbad", "dbab", "dbac", "dcdb", "dcbd",
  "dcbc", "dcba", "dcda", "dcad", "dcac", "dcab", "dadb",
  "dabd", "daba", "dabc", "dadc", "dacd", "daca", "dacb"},
 {"bcbd", "bcdb", "bcdc", "bcda", "bcba", "bcab", "bcac",
  "bcad", "bdbc", "bdcb", "bdcd", "bdca", "bdba",
  "bdab", "bdad", "bdac", "babc", "bacb", "baca", "bacd",
  "babd", "badb", "bada", "badc", "cbcd", "cbdc", "cbdb",
  "cbda", "cbca", "cbac", "cbab", "cbad", "cdcb", "cdbc",
  "cdbd", "cdba", "cdca", "cdac", "cdad", "cdab", "cacb",
  "cabc", "caba", "cabd", "cacd", "cadc", "cada", "cadb",
  "dbdc", "dbcd", "dbcb", "dbca", "dbda", "dbad", "dbab",
  "dbac", "dcdb", "dcbd", "dcbc", "dcba", "dcda", "dcad",
  "dcac", "dcab", "dadb", "dabd", "daba", "dabc", "dadc",
  "dacd", "daca", "dacb", "abac", "abca", "abcb", "abcd",
  "abad", "abda", "abdb", "abdc", "acab", "acba", "acbc",
  "acbd", "acad", "acda", "acdc", "acdb", "adab", "adba",
  "adbd", "adbc", "adac", "adca", "adcd", "adcb"},
 {"bdba", "bdab", "bdad", "bdac", "bdbc", "bdcb", "bdcd",
  "bdca", "babd", "badb", "bada", "badc", "babc",
  "bacb", "baca", "bacd", "bcbd", "bcdb", "bcdc", "bcda",
  "bcba", "bcab", "bcac", "bcad", "dbda", "dbad", "dbab",
  "dbac", "dbdc", "dbcd", "dbcb", "dbca", "dadb", "dabd",
  "daba", "dabc", "dadc", "dacd", "daca", "dacb", "dcdb",
  "dcbd", "dcbc", "dcba", "dcda", "dcad", "dcac", "dcab",
  "abad", "abda", "abdb", "abdc", "abac", "abca", "abcb",
  "abcd", "adab", "adba", "adbd", "adbc", "adac", "adca",
  "adcd", "adcb", "acab", "acba", "acbc", "acbd", "acad",
  "acda", "acdc", "acdb", "cbcd", "cbdc", "cbdb", "cbda",
  "cbca", "cbac", "cbab", "cbad", "cdcb", "cdbc", "cdbd",
  "cdba", "cdca", "cdac", "cdad", "cdab", "cacb", "cabc",
  "caba", "cabd", "cacd", "cadc", "cada", "cadb"},
 {"bdbc", "bdcb", "bdcd", "bdca", "bdba", "bdab", "bdad",
  "bdac", "bcbd", "bcdb", "bcdc", "bcda", "bcba",
  "bcab", "bcac", "bcad", "babd", "badb", "bada", "badc",
```

```
  "babc", "bacb", "baca", "bacd", "dbdc", "dbcd", "dbcb",
  "dbca", "dbda", "dbad", "dbab", "dbac", "dcdb", "dcbd",
  "dcbc", "dcba", "dcda", "dcad", "dcac", "dcab", "dadb",
  "dabd", "daba", "dabc", "dadc", "dacd", "daca", "dacb",
  "cbcd", "cbdc", "cbdb", "cbda", "cbca", "cbac", "cbab",
  "cbad", "cdcb", "cdbc", "cdbd", "cdba", "cdca", "cdac",
  "cdad", "cdab", "cacb", "cabc", "caba", "cabd", "cacd",
  "cadc", "cada", "cadb", "abad", "abda", "abdb", "abdc",
  "abac", "abca", "abcb", "abcd", "adab", "adba", "adbd",
  "adbc", "adac", "adca", "adcd", "adcb", "acab", "acba",
  "acbc", "acbd", "acad", "acda", "acdc", "acdb"},
 {"cacb", "cabc", "caba", "cabd", "cacd", "cadc", "cada",
  "cadb", "cbca", "cbac", "cbab", "cbad", "cbcd",
  "cbdc", "cbdb", "cbda", "cdca", "cdac", "cdad", "cdab",
  "cdcb", "cdbc", "cdbd", "cdba", "acab", "acba", "acbc",
  "acbd", "acad", "acda", "acdc", "acdb", "abac", "abca",
  "abcb", "abcd", "abad", "abda", "abdb", "abdc", "adac",
  "adca", "adcd", "adcb", "adab", "adba", "adbd", "adbc",
  "bcba", "bcab", "bcac", "bcad", "bcbd", "bcdb", "bcdc",
  "bcda", "babc", "bacb", "baca", "bacd", "babd", "badb",
  "bada", "badc", "bdbc", "bdcb", "bdcd", "bdca", "bdba",
  "bdab", "bdad", "bdac", "dcda", "dcad", "dcac", "dcab",
  "dcdb", "dcbd", "dcbc", "dcba", "dadc", "dacd", "daca",
  "dacb", "dadb", "dabd", "daba", "dabc", "dbdc", "dbcd",
  "dbcb", "dbca", "dbda", "dbad", "dbab", "dbac"},
 {"cacd", "cadc", "cada", "cadb", "cacb", "cabc", "caba",
  "cabd", "cdca", "cdac", "cdad", "cdab", "cdcb",
  "cdbc", "cdbd", "cdba", "cbca", "cbac", "cbab", "cbad",
  "cbcd", "cbdc", "cbdb", "cbda", "acad", "acda", "acdc",
  "acdb", "acab", "acba", "acbc", "acbd", "adac", "adca",
  "adcd", "adcb", "adab", "adba", "adbd", "adbc", "abac",
  "abca", "abcb", "abcd", "abad", "abda", "abdb", "abdc",
  "dcda", "dcad", "dcac", "dcab", "dcdb", "dcbd", "dcbc",
  "dcba", "dadc", "dacd", "daca", "dacb", "dadb", "dabd",
  "daba", "dabc", "dbdc", "dbcd", "dbcb", "dbca", "dbda",
  "dbad", "dbab", "dbac", "bcba", "bcab", "bcac", "bcad",
  "bcbd", "bcdb", "bcdc", "bcda", "babc", "bacb", "baca",
  "bacd", "babd", "badb", "bada", "badc", "bdbc", "bdcb",
  "bdcd", "bdca", "bdba", "bdab", "bdad", "bdac"},
 {"cbca", "cbac", "cbab", "cbad", "cbcd", "cbdc", "cbdb",
  "cbda", "cacb", "cabc", "caba", "cabd", "cacd",
  "cadc", "cada", "cadb", "cdcb", "cdbc", "cdbd", "cdba",
  "cdca", "cdac", "cdad", "cdab", "bcba", "bcab", "bcac",
  "bcad", "bcbd", "bcdb", "bcdc", "bcda", "babc", "bacb",
  "baca", "bacd", "babd", "badb", "bada", "badc", "bdbc",
  "bdcb", "bdcd", "bdca", "bdba", "bdab", "bdad", "bdac",
  "acab", "acba", "acbc", "acbd", "acad", "acda", "acdc",
  "acdb", "abac", "abca", "abcb", "abcd", "abad", "abda",
  "abdb", "abdc", "adac", "adca", "adcd", "adcb", "adab",
  "adba", "adbd", "adbc", "dcdb", "dcbd", "dcbc", "dcba",
  "dcda", "dcad", "dcac", "dcab", "dbdc", "dbcd", "dbcb",
```

```
 "dbca", "dbda", "dbad", "dbab", "dbac", "dadc", "dacd",
 "daca", "dacb", "dadb", "dabd", "daba", "dabc"},
{"cbcd", "cbdc", "cbdb", "cbda", "cbca", "cbac", "cbab",
 "cbad", "cdcb", "cdbc", "cdbd", "cdba", "cdca",
 "cdac", "cdad", "cdab", "cacb", "cabc", "caba", "cabd",
 "cacd", "cadc", "cada", "cadb", "bcbd", "bcdb", "bcdc",
 "bcda", "bcba", "bcab", "bcac", "bcad", "bdbc", "bdcb",
 "bdcd", "bdca", "bdba", "bdab", "bdad", "bdac", "babc",
 "bacb", "baca", "bacd", "babd", "badb", "bada", "badc",
 "dcdb", "dcbd", "dcbc", "dcba", "dcda", "dcad", "dcac",
 "dcab", "dbdc", "dbcd", "dbcb", "dbca", "dbda", "dbad",
 "dbab", "dbac", "dadc", "dacd", "daca", "dacb", "dadb",
 "dabd", "daba", "dabc", "acab", "acba", "acbc", "acbd",
 "acad", "acda", "acdc", "acdb", "abac", "abca", "abcb",
 "abcd", "abad", "abda", "abdb", "abdc", "adac", "adca",
 "adcd", "adcb", "adab", "adba", "adbd", "adbc"},
{"cdca", "cdac", "cdad", "cdab", "cdcb", "cdbc", "cdbd",
 "cdba", "cacd", "cadc", "cada", "cadb", "cacb",
 "cabc", "caba", "cabd", "cbcd", "cbdc", "cbdb", "cbda",
 "cbca", "cbac", "cbab", "cbad", "dcda", "dcad", "dcac",
 "dcab", "dcdb", "dcbd", "dcbc", "dcba", "dadc", "dacd",
 "daca", "dacb", "dadb", "dabd", "daba", "dabc", "dbdc",
 "dbcd", "dbcb", "dbca", "dbda", "dbad", "dbab", "dbac",
 "acad", "acda", "acdc", "acdb", "acab", "acba", "acbc",
 "acbd", "adac", "adca", "adcd", "adcb", "adab", "adba",
 "adbd", "adbc", "abac", "abca", "abcb", "abcd", "abad",
 "abda", "abdb", "abdc", "bcbd", "bcdb", "bcdc", "bcda",
 "bcba", "bcab", "bcac", "bcad", "bdbc", "bdcb", "bdcd",
 "bdca", "bdba", "bdab", "bdad", "bdac", "babc", "bacb",
 "baca", "bacd", "babd", "badb", "bada", "badc"},
{"cdcb", "cdbc", "cdbd", "cdba", "cdca", "cdac", "cdad",
 "cdab", "cbcd", "cbdc", "cbdb", "cbda", "cbca",
 "cbac", "cbab", "cbad", "cacd", "cadc", "cada", "cadb",
 "cacb", "cabc", "caba", "cabd", "dcdb", "dcbd", "dcbc",
 "dcba", "dcda", "dcad", "dcac", "dcab", "dbdc", "dbcd",
 "dbcb", "dbca", "dbda", "dbad", "dbab", "dbac", "dadc",
 "dacd", "daca", "dacb", "dadb", "dabd", "daba", "dabc",
 "bcbd", "bcdb", "bcdc", "bcda", "bcba", "bcab", "bcac",
 "bcad", "bdbc", "bdcb", "bdcd", "bdca", "bdba", "bdab",
 "bdad", "bdac", "babc", "bacb", "baca", "bacd", "babd",
 "badb", "bada", "badc", "acad", "acda", "acdc", "acdb",
 "acab", "acba", "acbc", "acbd", "adac", "adca", "adcd",
 "adcb", "adab", "adba", "adbd", "adbc", "abac", "abca",
 "abcb", "abcd", "abad", "abda", "abdb", "abdc"},
{"dadb", "dabd", "daba", "dabc", "dadc", "dacd", "daca",
 "dacb", "dbda", "dbad", "dbab", "dbac", "dbdc",
 "dbcd", "dbcb", "dbca", "dcda", "dcad", "dcac", "dcab",
 "dcdb", "dcbd", "dcbc", "dcba", "adab", "adba", "adbd",
 "adbc", "adac", "adca", "adcd", "adcb", "abad", "abda",
 "abdb", "abdc", "abac", "abca", "abcb", "abcd", "acad",
 "acda", "acdc", "acdb", "acab", "acba", "acbc", "acbd",
```

```
 "bdba", "bdab", "bdad", "bdac", "bdbc", "bdcb", "bdcd",
 "bdca", "babd", "badb", "bada", "badc", "babc", "bacb",
 "baca", "bacd", "bcbd", "bcdb", "bcdc", "bcda", "bcba",
 "bcab", "bcac", "bcad", "cdca", "cdac", "cdad", "cdab",
 "cdcb", "cdbc", "cdbd", "cdba", "cacd", "cadc", "cada",
 "cadb", "cacb", "cabc", "caba", "cabd", "cbcd", "cbdc",
 "cbdb", "cbda", "cbca", "cbac", "cbab", "cbad"},
{"dadc", "dacd", "daca", "dacb", "dadb", "dabd", "daba",
 "dabc", "dcda", "dcad", "dcac", "dcab", "dcdb",
 "dcbd", "dcbc", "dcba", "dbda", "dbad", "dbab", "dbac",
 "dbdc", "dbcd", "dbcb", "dbca", "adac", "adca", "adcd",
 "adcb", "adab", "adba", "adbd", "adbc", "acad", "acda",
 "acdc", "acdb", "acab", "acba", "acbc", "acbd", "abad",
 "abda", "abdb", "abdc", "abac", "abca", "abcb", "abcd",
 "cdca", "cdac", "cdad", "cdab", "cdcb", "cdbc", "cdbd",
 "cdba", "cacd", "cadc", "cada", "cadb", "cacb", "cabc",
 "caba", "cabd", "cbcd", "cbdc", "cbdb", "cbda", "cbca",
 "cbac", "cbab", "cbad", "bdba", "bdab", "bdad", "bdac",
 "bdbc", "bdcb", "bdcd", "bdca", "babd", "badb", "bada",
 "badc", "babc", "bacb", "baca", "bacd", "bcbd", "bcdb",
 "bcdc", "bcda", "bcba", "bcab", "bcac", "bcad"},
{"dbda", "dbad", "dbab", "dbac", "dbdc", "dbcd", "dbcb",
 "dbca", "dadb", "dabd", "daba", "dabc", "dadc",
 "dacd", "daca", "dacb", "dcdb", "dcbd", "dcbc", "dcba",
 "dcda", "dcad", "dcac", "dcab", "bdba", "bdab", "bdad",
 "bdac", "bdbc", "bdcb", "bdcd", "bdca", "babd", "badb",
 "bada", "badc", "babc", "bacb", "baca", "bacd", "bcbd",
 "bcdb", "bcdc", "bcda", "bcba", "bcab", "bcac", "bcad",
 "adab", "adba", "adbd", "adbc", "adac", "adca", "adcd",
 "adcb", "abad", "abda", "abdb", "abdc", "abac", "abca",
 "abcb", "abcd", "acad", "acda", "acdc", "acdb", "acab",
 "acba", "acbc", "acbd", "cdcb", "cdbc", "cdbd", "cdba",
 "cdca", "cdac", "cdad", "cdab", "cbcd", "cbdc", "cbdb",
 "cbda", "cbca", "cbac", "cbab", "cbad", "cacd", "cadc",
 "cada", "cadb", "cacb", "cabc", "caba", "cabd"},
{"dbdc", "dbcd", "dbcb", "dbca", "dbda", "dbad", "dbab",
 "dbac", "dcdb", "dcbd", "dcbc", "dcba", "dcda",
 "dcad", "dcac", "dcab", "dadb", "dabd", "daba", "dabc",
 "dadc", "dacd", "daca", "dacb", "bdbc", "bdcb", "bdcd",
 "bdca", "bdba", "bdab", "bdad", "bdac", "bcbd", "bcdb",
 "bcdc", "bcda", "bcba", "bcab", "bcac", "bcad", "babd",
 "badb", "bada", "badc", "babc", "bacb", "baca", "bacd",
 "cdcb", "cdbc", "cdbd", "cdba", "cdca", "cdac", "cdad",
 "cdab", "cbcd", "cbdc", "cbdb", "cbda", "cbca", "cbac",
 "cbab", "cbad", "cacd", "cadc", "cada", "cadb", "cacb",
 "cabc", "caba", "cabd", "adab", "adba", "adbd", "adbc",
 "adac", "adca", "adcd", "adcb", "abad", "abda", "abdb",
 "abdc", "abac", "abca", "abcb", "abcd", "acad", "acda",
 "acdc", "acdb", "acab", "acba", "acbc", "acbd"},
{"dcda", "dcad", "dcac", "dcab", "dcdb", "dcbd", "dcbc",
 "dcba", "dadc", "dacd", "daca", "dacb", "dadb",
```

```
      "dabd", "daba", "dabc", "dbdc", "dbcd", "dbcb", "dbca",
      "dbda", "dbad", "dbab", "dbac", "cdca", "cdac", "cdad",
      "cdab", "cdcb", "cdbc", "cdbd", "cdba", "cacd", "cadc",
      "cada", "cadb", "cacb", "cabc", "caba", "cabd", "cbcd",
      "cbdc", "cbdb", "cbda", "cbca", "cbac", "cbab", "cbad",
      "adac", "adca", "adcd", "adcb", "adab", "adba", "adbd",
      "adbc", "acad", "acda", "acdc", "acdb", "acab", "acba",
      "acbc", "acbd", "abad", "abda", "abdb", "abdc", "abac",
      "abca", "abcb", "abcd", "bdbc", "bdcb", "bdcd", "bdca",
      "bdba", "bdab", "bdad", "bdac", "bcbd", "bcdb", "bcdc",
      "bcda", "bcba", "bcab", "bcac", "bcad", "babd", "badb",
      "bada", "badc", "babc", "bacb", "baca", "bacd"},
     {"dcdb", "dcbd", "dcbc", "dcba", "dcda", "dcad", "dcac",
      "dcab", "dbdc", "dbcd", "dbcb", "dbca", "dbda",
      "dbad", "dbab", "dbac", "dadc", "dacd", "daca", "dacb",
      "dadb", "dabd", "daba", "dabc", "cdcb", "cdbc", "cdbd",
      "cdba", "cdca", "cdac", "cdad", "cdab", "cbcd", "cbdc",
      "cbdb", "cbda", "cbca", "cbac", "cbab", "cbad", "cacd",
      "cadc", "cada", "cadb", "cacb", "cabc", "caba", "cabd",
      "bdbc", "bdcb", "bdcd", "bdca", "bdba", "bdab", "bdad",
      "bdac", "bcbd", "bcdb", "bcdc", "bcda", "bcba", "bcab",
      "bcac", "bcad", "babd", "badb", "bada", "badc", "babc",
      "bacb", "baca", "bacd", "adac", "adca", "adcd", "adcb",
      "adab", "adba", "adbd", "adbc", "acad", "acda", "acdc",
      "acdb", "acab", "acba", "acbc", "acbd", "abad", "abda",
      "abdb", "abdc", "abac", "abca", "abcb", "abcd"}};

  Clear[permRuleForStrings];
permRuleForStrings[n_] :=
   Thread[Rule[allWords4ForRules[[1]], allWords4ForRules[[n]]]];

  Clear[permRuleForExpressions];
permRuleForExpressions[n_] :=
   Rule[ToExpression[#[[1]]], ToExpression[#[[2]]]] & /@
    permRuleForStrings[n];

  Clear[€permRule];
€permRule[1] = {};
(€permRule[#] = permRuleForExpressions[#]) & /@
   Range[2, Length[allWords4ForRules]];

  reducedWordsList4 = {"abac", "abca", "abcb", "abcd"};

  allWordsList4 =
   StringReplace[reducedWordsList4, #] & /@ listOfRules;

  allSymbolsList4 = (ToExpression /@ #) & /@ allWordsList4;

allCumulIntegersList4 =
   allSymbolsList4 /. €ruleSymbolsToCumulIntegerLists;
```

```
€ruleCumulIntegerListsToSymbols =
    {{0, 1, 1, 1025} → abac, {0, 1, 1025, 1025} → abca,
     {0, 1, 1025, 1026} → abcb, {0, 1, 1025, 1 049 601} → abcd,
     {0, 1, 1, 1 048 577} → abad, {0, 1, 1 048 577, 1 048 577} → abda,
     {0, 1, 1 048 577, 1 048 578} → abdb, {0, 1, 1 048 577, 1 049 601} →
      abdc, {0, 1024, 1024, 1025} → acab, {0, 1024, 1025, 1025} → acba,
     {0, 1024, 1025, 2049} → acbc, {0, 1024, 1025, 1 049 601} → acbd,
     {0, 1024, 1024, 1 049 600} → acad, {0, 1024, 1 049 600, 1 049 600} →
      acda, {0, 1024, 1 049 600, 1 050 624} → acdc,
     {0, 1024, 1 049 600, 1 049 601} → acdb,
     {0, 1 048 576, 1 048 576, 1 048 577} → adab,
     {0, 1 048 576, 1 048 577, 1 048 577} → adba,
     {0, 1 048 576, 1 048 577, 2 097 153} → adbd,
     {0, 1 048 576, 1 048 577, 1 049 601} → adbc,
     {0, 1 048 576, 1 048 576, 1 049 600} → adac,
     {0, 1 048 576, 1 049 600, 1 049 600} → adca,
     {0, 1 048 576, 1 049 600, 2 098 176} → adcd,
     {0, 1 048 576, 1 049 600, 1 049 601} → adcb,
     {1, 1, 2, 1026} → babc, {1, 1, 1025, 1026} → bacb,
     {1, 1, 1025, 1025} → baca, {1, 1, 1025, 1 049 601} → bacd,
     {1, 1, 2, 1 048 578} → babd, {1, 1, 1 048 577, 1 048 578} → badb,
     {1, 1, 1 048 577, 1 048 577} → bada,
     {1, 1, 1 048 577, 1 049 601} → badc,
     {1, 1025, 1026, 1026} → bcba, {1, 1025, 1025, 1026} → bcab,
     {1, 1025, 1025, 2049} → bcac, {1, 1025, 1025, 1 049 601} → bcad,
     {1, 1025, 1026, 1 049 602} → bcbd, {1, 1025, 1 049 601, 1 049 602} →
      bcdb, {1, 1025, 1 049 601, 1 050 625} → bcdc,
     {1, 1025, 1 049 601, 1 049 601} → bcda,
     {1, 1 048 577, 1 048 578, 1 048 578} → bdba,
     {1, 1 048 577, 1 048 577, 1 048 578} → bdab,
     {1, 1 048 577, 1 048 577, 2 097 153} → bdad,
     {1, 1 048 577, 1 048 577, 1 049 601} → bdac,
     {1, 1 048 577, 1 048 578, 1 049 602} → bdbc,
     {1, 1 048 577, 1 049 601, 1 049 602} → bdcb,
     {1, 1 048 577, 1 049 601, 2 098 177} → bdcd,
     {1, 1 048 577, 1 049 601, 1 049 601} → bdca,
     {1024, 1024, 2048, 2049} → cacb, {1024, 1024, 1025, 2049} → cabc,
     {1024, 1024, 1025, 1025} → caba, {1024, 1024, 1025, 1 049 601} →
      cabd, {1024, 1024, 2048, 1 050 624} → cacd,
     {1024, 1024, 1 049 600, 1 050 624} → cadc,
     {1024, 1024, 1 049 600, 1 049 600} → cada,
     {1024, 1024, 1 049 600, 1 049 601} → cadb,
     {1024, 1025, 2049, 2049} → cbca, {1024, 1025, 1025, 2049} → cbac,
     {1024, 1025, 1025, 1026} → cbab, {1024, 1025, 1025, 1 049 601} →
      cbad, {1024, 1025, 2049, 1 050 625} → cbcd,
     {1024, 1025, 1 049 601, 1 050 625} → cbdc,
     {1024, 1025, 1 049 601, 1 049 602} → cbdb,
     {1024, 1025, 1 049 601, 1 049 601} → cbda,
     {1024, 1 049 600, 1 050 624, 1 050 624} → cdca,
     {1024, 1 049 600, 1 049 600, 1 050 624} → cdac,
```

```
    {1024, 1 049 600, 1 049 600, 2 098 176} → cdad,
    {1024, 1 049 600, 1 049 600, 1 049 601} → cdab,
    {1024, 1 049 600, 1 050 624, 1 050 625} → cdcb,
    {1024, 1 049 600, 1 049 601, 1 050 625} → cdbc,
    {1024, 1 049 600, 1 049 601, 2 098 177} → cdbd,
    {1024, 1 049 600, 1 049 601, 1 049 601} → cdba,
    {1 048 576, 1 048 576, 2 097 152, 2 097 153} → dadb,
    {1 048 576, 1 048 576, 1 048 577, 2 097 153} → dabd,
    {1 048 576, 1 048 576, 1 048 577, 1 048 577} → daba,
    {1 048 576, 1 048 576, 1 048 577, 1 049 601} → dabc,
    {1 048 576, 1 048 576, 2 097 152, 2 098 176} → dadc,
    {1 048 576, 1 048 576, 1 049 600, 2 098 176} → dacd,
    {1 048 576, 1 048 576, 1 049 600, 1 049 600} → daca,
    {1 048 576, 1 048 576, 1 049 600, 1 049 601} → dacb,
    {1 048 576, 1 048 577, 2 097 153, 2 097 153} → dbda,
    {1 048 576, 1 048 577, 1 048 577, 2 097 153} → dbad,
    {1 048 576, 1 048 577, 1 048 577, 1 048 578} → dbab,
    {1 048 576, 1 048 577, 1 048 577, 1 049 601} → dbac,
    {1 048 576, 1 048 577, 2 097 153, 2 098 177} → dbdc,
    {1 048 576, 1 048 577, 1 049 601, 2 098 177} → dbcd,
    {1 048 576, 1 048 577, 1 049 601, 1 049 602} → dbcb,
    {1 048 576, 1 048 577, 1 049 601, 1 049 601} → dbca,
    {1 048 576, 1 049 600, 2 098 176, 2 098 176} → dcda,
    {1 048 576, 1 049 600, 1 049 600, 2 098 176} → dcad,
    {1 048 576, 1 049 600, 1 049 600, 1 050 624} → dcac,
    {1 048 576, 1 049 600, 1 049 600, 1 049 601} → dcab,
    {1 048 576, 1 049 600, 2 098 176, 2 098 177} → dcdb,
    {1 048 576, 1 049 600, 1 049 601, 2 098 177} → dcbd,
    {1 048 576, 1 049 600, 1 049 601, 1 050 625} → dcbc,
    {1 048 576, 1 049 600, 1 049 601, 1 049 601} → dcba};

  Clear[constrFu, ordinalIndexForCumulIntegerList4];
constrFu[n_] := (ordinalIndexForCumulIntegerList4[#] :=
       Position[allCumulIntegersList4[[n]], #][[1, 1]]) & /@
    allCumulIntegersList4[[n]];

constrFu[#] & /@ Range[Length[allCumulIntegersList4]];

  Clear[constructFu, tryProperExtensionVis];
constructFu[n_] := (tryProperExtensionVis[#, index_] :=
       €reducedWordListSuff4Add4ToExpressions[[
         ordinalIndexForCumulIntegerList4[#], 2, index]] /.
        €permRule[n]) & /@ allCumulIntegersList4[[n]];

constructFu[#] & /@ Range[Length[allCumulIntegersList4]];

Clear[constructFuVisWholeList, tryProperExtensionVisList];
constructFuVisWholeList[n_] := (tryProperExtensionVisList[#] :=
```

```
        €reducedWordListSuff4Add4ToExpressions[[
          ordinalIndexForCumulIntegerList4[#], 2]] /.
         €permRule[n]) & /@ allCumulIntegersList4[[n]];

  constructFuVisWholeList[#] & /@
   Range[Length[allCumulIntegersList4]];

  Clear[tryProperExtension];
tryProperExtension[x_, index_] :=
   (tryProperExtensionVis[x, index] /.
     €ruleSymbolsToCumulIntegerLists);
tryProperExtension[x_] := (tryProperExtensionVisList[x] /.
     €ruleSymbolsToCumulIntegerLists);

  Clear[lengthOfListOfCatenableCumIntLis];
lengthOfListOfCatenableCumIntLis[x_List] :=
   Length[€reducedWordListSuff4Add4ToExpressions[[
     ordinalIndexForCumulIntegerList4[x], 2]]];)
```

The program in the package `alphThreeLetIntCase.m`

```
(*************************************************************
  ****This file was generated automatically by the Mathematica
 front end. It contains Initialization cells from a Notebook file,
which typically will have the same name as this file except ending
 in ".nb" instead of ".m". This file is intended to be loaded
 into the Mathematica kernel using the package loading commands
 Get or Needs. Doing so is equivalent to using the Evaluate
 Initialization Cells menu command in the front end. DO NOT EDIT
 THIS FILE. This entire file is regenerated automatically each
 time the parent Notebook file is saved in the Mathematica front
 end. Any changes you make to this file will be overwritten.
*************************************************************)
 *******)Off[General::spell]
alphThreeLetIntCase := (Off[General::spell];
Off[General::spell1];

  Clear[cutPref];
cutPref[{x_String, y_String}] := StringCases[x <> "." <> y,
     (p___ ~~ s1__ ~~ "." ~~ p___ ~~ s2__ → s2)][[1]];

  Clear[formPrefSuffPairs];
formPrefSuffPairs[{x_String, y_String}] :=
   StringCases[x, {p__ ~~ y → {p, y}, y → {x, ""}}][[1]];

  Clear[selectSuffixes];
selectSuffixes[pref_String, word_List] :=
```

```
    {pref, StringCases[#, StartOfString ~~ pref ~~
            x___ ~~ EndOfString → x] & /@ word // Flatten};
  (*Save["reducedAddWordList8.txt",reducedAddWordList8];
  Save["extendWordList4by4.txt",extendWordList4by4];*)

  Clear[cutPref];
cutPref[{x_String, y_String}] := StringCases[x <> "." <> y,
     (p___ ~~ s1__ ~~ "." ~~ p___ ~~ s2__ → s2)][[1]];

  Clear[formPrefSuffPairs];
formPrefSuffPairs[{x_String, y_String}] :=
   StringCases[x, {p__ ~~ y → {p, y}, y → {x, ""}}][[1]];

  €reducedWordListSuff4Add4ToExpressions =
   {{aaab, {aaac, aaca, aacb, aacc, acaa, acab, acba, acbb, acbc,
      accb, accc, bbaa, bbac, bbca, bbcb, bbcc, bcaa, bcab,
      bcac, bcba, bcbb, bcca, bccc, caaa, caab, caba, cabb,
      cacc, cbaa, cbab, cbba, cbbb, ccaa, ccac, ccca, cccb}},
    {aaba, {aaca, aacb, aacc, acaa, acab, acba, acbb, acbc,
      accb, accc, caaa, caab, caba, cabb, cbaa, cbab,
      cbba, cbbb, cbcc, ccbb, ccbc, ccca, cccb}},
    {aabb, {baaa, baac, baca, bacb, bacc, bcaa, bcab, bcac, bcba,
      bcbb, bcca, bccc, caaa, caab, caba, cabb, cabc, cacb, cacc,
      cbaa, cbab, cbac, cbba, cbbb, ccaa, ccab, ccac, ccca, cccb}},
    {aabc, {aaab, aaac, aaba, aabb, abaa, abbb, abbc, accb,
      accc, baaa, babb, babc, bbaa, bbab, bbac, bbba,
      caaa, cacb, cacc, ccaa, ccab, ccac, ccba, ccbb}},
    {abaa, {acaa, acab, acba, acbb, acbc, accb, accc, caaa,
      caab, caba, cabb, cabc, cbaa, cbab, cbac, cbba,
      cbbb, cbca, cbcc, ccba, ccbb, ccbc, ccca, cccb}},
    {abac, {aaab, aaba, aabb, abaa, abbb, abbc, baaa, babb,
      bbaa, bbab, bbba, bbbc, bcca, bccc, cbba, cbbb,
      cbca, cbcc, ccaa, ccab, ccba, ccbb, ccbc}},
    {abbb, {aaab, aaac, aaca, aacb, aacc, acaa, acab, acba, acbb,
      acbc, accb, accc, caaa, caab, caba, cabb, cabc, cacb, cacc,
      cbaa, cbab, cbac, cbba, cbbb, ccaa, ccab, ccac, ccca, cccb}},
    {abbc, {aaab, aaac, aaba, aabb, abaa, abbb, abcc, acbc, accb,
      accc, baaa, baac, babb, baca, bacc, bbaa, bbab, bbba, caaa,
      caab, caba, cabc, cacb, cacc, ccaa, ccab, ccac, ccba, ccbb}},
    {abca, {aaba, aabb, aacc, abaa, abbb, abbc, baaa, bbba,
      bbbc, bbcb, bbcc, ccba, ccbb, ccbc, cccb}},
    {abcb, {aaab, aaac, aaca, aacc, abbb, abcc, baaa,
      baac, babb, baca, bacc, bbaa, bbab}},
    {abcc, {aaab, aaac, aaba, aabb, aabc, acba, acbb, accc,
      caaa, caab, caba, cabb, cacc, cbaa, cbab, cbba, cbbb}}};

  €ruleSymbolsToCumulIntegerLists =
   Thread[Rule[wList4InSymbols, wList4CumulIntegerLists]];
```

```
€ruleSymbolsToCumulIntegerLists = {aaab → {0, 0, 0, 1},
    aaba → {0, 0, 1, 1}, aabb → {0, 0, 1, 2}, aabc → {0, 0, 1, 65 537},
    abaa → {0, 1, 1, 1}, abac → {0, 1, 1, 65 537},
    abbb → {0, 1, 2, 3}, abbc → {0, 1, 2, 65 538},
    abca → {0, 1, 65 537, 65 537}, abcb → {0, 1, 65 537, 65 538},
    abcc → {0, 1, 65 537, 131 073}, aaac → {0, 0, 0, 65 536},
    aaca → {0, 0, 65 536, 65 536}, aacc → {0, 0, 65 536, 131 072},
    aacb → {0, 0, 65 536, 65 537}, acaa → {0, 65 536, 65 536, 65 536},
    acab → {0, 65 536, 65 536, 65 537},
    accc → {0, 65 536, 131 072, 196 608},
    accb → {0, 65 536, 131 072, 131 073}, acba →
     {0, 65 536, 65 537, 65 537}, acbc → {0, 65 536, 65 537, 131 073},
    acbb → {0, 65 536, 65 537, 65 538}, bbba → {1, 2, 3, 3},
    bbab → {1, 2, 2, 3}, bbaa → {1, 2, 2, 2}, bbac → {1, 2, 2, 65 538},
    babb → {1, 1, 2, 3}, babc → {1, 1, 2, 65 538},
    baaa → {1, 1, 1, 1}, baac → {1, 1, 1, 65 537},
    bacb → {1, 1, 65 537, 65 538}, baca → {1, 1, 65 537, 65 537},
    bacc → {1, 1, 65 537, 131 073}, bbbc → {1, 2, 3, 65 539},
    bbcb → {1, 2, 65 538, 65 539}, bbcc → {1, 2, 65 538, 131 074},
    bbca → {1, 2, 65 538, 65 538}, bcbb → {1, 65 537, 65 538, 65 539},
    bcba → {1, 65 537, 65 538, 65 538}, bccc →
     {1, 65 537, 131 073, 196 609}, bcca → {1, 65 537, 131 073, 131 073},
    bcab → {1, 65 537, 65 537, 65 538}, bcac →
     {1, 65 537, 65 537, 131 073}, bcaa → {1, 65 537, 65 537, 65 537},
    ccca → {65 536, 131 072, 196 608, 196 608},
    ccac → {65 536, 131 072, 131 072, 196 608},
    ccaa → {65 536, 131 072, 131 072, 131 072},
    ccab → {65 536, 131 072, 131 072, 131 073},
    cacc → {65 536, 65 536, 131 072, 196 608},
    cacb → {65 536, 65 536, 131 072, 131 073},
    caaa → {65 536, 65 536, 65 536, 65 536},
    caab → {65 536, 65 536, 65 536, 65 537},
    cabc → {65 536, 65 536, 65 537, 131 073},
    caba → {65 536, 65 536, 65 537, 65 537},
    cabb → {65 536, 65 536, 65 537, 65 538},
    cccb → {65 536, 131 072, 196 608, 196 609},
    ccbc → {65 536, 131 072, 131 073, 196 609},
    ccbb → {65 536, 131 072, 131 073, 131 074},
    ccba → {65 536, 131 072, 131 073, 131 073},
    cbcc → {65 536, 65 537, 131 073, 196 609},
    cbca → {65 536, 65 537, 131 073, 131 073},
    cbbb → {65 536, 65 537, 65 538, 65 539},
    cbba → {65 536, 65 537, 65 538, 65 538},
    cbac → {65 536, 65 537, 65 537, 131 073},
    cbab → {65 536, 65 537, 65 537, 65 538},
    cbaa → {65 536, 65 537, 65 537, 65 537}};

  listOfRules = {{"a" → "a", "b" → "b", "c" → "c"},
    {"a" → "a", "b" → "c", "c" → "b"}, {"a" → "b", "b" → "a",
     "c" → "c"}, {"a" → "b", "b" → "c", "c" → "a"}, {"a" → "c",
     "b" → "a", "c" → "b"}, {"a" → "c", "b" → "b", "c" → "a"}};
```

```
allWords4ForRules =
   {{"aaab", "aaba", "aabb", "aabc", "abaa", "abac", "abbb", "abbc",
     "abca", "abcb", "abcc", "aaac", "aaca", "aacc", "aacb", "acaa",
     "acab", "accc", "accb", "acba", "acbc", "acbb", "bbba", "bbab",
     "bbaa", "bbac", "babb", "babc", "baaa", "baac", "bacb",
     "baca", "bacc", "bbbc", "bbcb", "bbcc", "bbca", "bcbb",
     "bcba", "bccc", "bcca", "bcab", "bcac", "bcaa", "ccca",
     "ccac", "ccaa", "ccab", "cacc", "cacb", "caaa", "caab",
     "cabc", "caba", "cabb", "cccb", "ccbc", "ccbb", "ccba",
     "cbcc", "cbca", "cbbb", "cbba", "cbac", "cbab", "cbaa"},
    {"aaac", "aaca", "aacc", "aacb", "acaa", "acab", "accc",
     "accb", "acba", "acbc", "acbb", "aaab", "aaba", "aabb", "aabc",
     "abaa", "abac", "abbb", "abbc", "abca", "abcb", "abcc", "ccca",
     "ccac", "ccaa", "ccab", "cacc", "cacb", "caaa", "caab", "cabc",
     "caba", "cabb", "cccb", "ccbc", "ccbb", "ccba", "cbcc",
     "cbca", "cbbb", "cbba", "cbac", "cbab", "cbaa", "bbba",
     "bbab", "bbaa", "bbac", "babb", "babc", "baaa", "baac",
     "bacb", "baca", "bacc", "bbbc", "bbcb", "bbcc", "bbca",
     "bcbb", "bcba", "bccc", "bcca", "bcab", "bcac", "bcaa"},
    {"bbba", "bbab", "bbaa", "bbac", "babb", "babc", "baaa",
     "baac", "bacb", "baca", "bacc", "bbbc", "bbcb", "bbcc", "bbca",
     "bcbb", "bcba", "bccc", "bcca", "bcab", "bcac", "bcaa", "aaab",
     "aaba", "aabb", "aabc", "abaa", "abac", "abbb", "abbc", "abca",
     "abcb", "abcc", "aaac", "aaca", "aacc", "aacb", "acaa",
     "acab", "accc", "accb", "acba", "acbc", "acbb", "cccb",
     "ccbc", "ccbb", "ccba", "cbcc", "cbca", "cbbb", "cbba",
     "cbac", "cbab", "cbaa", "ccca", "ccac", "ccaa", "ccab",
     "cacc", "cacb", "caaa", "caab", "cabc", "caba", "cabb"},
    {"bbbc", "bbcb", "bbcc", "bbca", "bcbb", "bcba", "bccc",
     "bcca", "bcab", "bcac", "bcaa", "bbba", "bbab", "bbaa", "bbac",
     "babb", "babc", "baaa", "baac", "bacb", "baca", "bacc", "cccb",
     "ccbc", "ccbb", "ccba", "cbcc", "cbca", "cbbb", "cbba", "cbac",
     "cbab", "cbaa", "ccca", "ccac", "ccaa", "ccab", "cacc",
     "cacb", "caaa", "caab", "cabc", "caba", "cabb", "aaab",
     "aaba", "aabb", "aabc", "abaa", "abac", "abbb", "abbc",
     "abca", "abcb", "abcc", "aaac", "aaca", "aacc", "aacb",
     "acaa", "acab", "accc", "accb", "acba", "acbc", "acbb"},
    {"ccca", "ccac", "ccaa", "ccab", "cacc", "cacb", "caaa",
     "caab", "cabc", "caba", "cabb", "cccb", "ccbc", "ccbb", "ccba",
     "cbcc", "cbca", "cbbb", "cbba", "cbac", "cbab", "cbaa", "aaac",
     "aaca", "aacc", "aacb", "acaa", "acab", "accc", "accb", "acba",
     "acbc", "acbb", "aaab", "aaba", "aabb", "aabc", "abaa",
     "abac", "abbb", "abbc", "abca", "abcb", "abcc", "bbbc",
     "bbcb", "bbcc", "bbca", "bcbb", "bcba", "bccc", "bcca",
     "bcab", "bcac", "bcaa", "bbba", "bbab", "bbaa", "bbac",
     "babb", "babc", "baaa", "baac", "bacb", "baca", "bacc"},
    {"cccb", "ccbc", "ccbb", "ccba", "cbcc", "cbca", "cbbb",
     "cbba", "cbac", "cbab", "cbaa", "ccca", "ccac", "ccaa", "ccab",
     "cacc", "cacb", "caaa", "caab", "cabc", "caba", "cabb", "bbbc",
     "bbcb", "bbcc", "bbca", "bcbb", "bcba", "bccc", "bcca", "bcab",
```

```
      "bcac", "bcaa", "bbba", "bbab", "bbaa", "bbac", "babb",
      "babc", "baaa", "baac", "bacb", "baca", "bacc", "aaac",
      "aaca", "aacc", "aacb", "acaa", "acab", "accc", "accb",
      "acba", "acbc", "acbb", "aaab", "aaba", "aabb", "aabc",
      "abaa", "abac", "abbb", "abbc", "abca", "abcb", "abcc"}};

  Clear[permRuleForStrings];
permRuleForStrings[n_] :=
   Thread[Rule[allWords4ForRules[[1]], allWords4ForRules[[n]]]];

Clear[permRuleForExpressions];
permRuleForExpressions[n_] :=
   Rule[ToExpression[#[[1]]], ToExpression[#[[2]]]] & /@
    permRuleForStrings[n];

  Clear[€permRule];
€permRule[1] = {};
(€permRule[#] = permRuleForExpressions[#]) & /@
   Range[2, Length[allWords4ForRules]];

  reducedWordsList4 = {"aaab", "aaba", "aabb", "aabc",
    "abaa", "abac", "abbb", "abbc", "abca", "abcb", "abcc"};

  allWordsList4 =
   StringReplace[reducedWordsList4, #] & /@ listOfRules;

  allSymbolsList4 = (ToExpression /@ #) & /@ allWordsList4;

  allCumulIntegersList4 =
   allSymbolsList4 /. €ruleSymbolsToCumulIntegerLists;

  €ruleCumulIntegerListsToSymbols = {{0, 0, 0, 1} → aaab,
    {0, 0, 1, 1} → aaba, {0, 0, 1, 2} → aabb, {0, 0, 1, 65 537} → aabc,
    {0, 1, 1, 1} → abaa, {0, 1, 1, 65 537} → abac, {0, 1, 2, 3} → abbb,
    {0, 1, 2, 65 538} → abbc, {0, 1, 65 537, 65 537} → abca,
    {0, 1, 65 537, 65 538} → abcb, {0, 1, 65 537, 131 073} → abcc,
    {0, 0, 0, 65 536} → aaac, {0, 0, 65 536, 65 536} → aaca,
    {0, 0, 65 536, 131 072} → aacc, {0, 0, 65 536, 65 537} → aacb,
    {0, 65 536, 65 536, 65 536} → acaa, {0, 65 536, 65 536, 65 537} →
     acab, {0, 65 536, 131 072, 196 608} → accc,
    {0, 65 536, 131 072, 131 073} → accb, {0, 65 536, 65 537, 65 537} →
     acba, {0, 65 536, 65 537, 131 073} → acbc,
    {0, 65 536, 65 537, 65 538} → acbb, {1, 2, 3, 3} → bbba,
    {1, 2, 2, 3} → bbab, {1, 2, 2, 2} → bbaa, {1, 2, 2, 65 538} → bbac,
    {1, 1, 2, 3} → babb, {1, 1, 2, 65 538} → babc,
    {1, 1, 1, 1} → baaa, {1, 1, 1, 65 537} → baac,
    {1, 1, 65 537, 65 538} → bacb, {1, 1, 65 537, 65 537} → baca,
    {1, 1, 65 537, 131 073} → bacc, {1, 2, 3, 65 539} → bbbc,
    {1, 2, 65 538, 65 539} → bbcb, {1, 2, 65 538, 131 074} → bbcc,
    {1, 2, 65 538, 65 538} → bbca, {1, 65 537, 65 538, 65 539} → bcbb,
```

```
    {1, 65 537, 65 538, 65 538} → bcba,
    {1, 65 537, 131 073, 196 609} → bccc, {1, 65 537, 131 073, 131 073} →
     bcca, {1, 65 537, 65 537, 65 538} → bcab,
    {1, 65 537, 65 537, 131 073} → bcac, {1, 65 537, 65 537, 65 537} →
     bcaa, {65 536, 131 072, 196 608, 196 608} → ccca,
    {65 536, 131 072, 131 072, 196 608} → ccac,
    {65 536, 131 072, 131 072, 131 072} → ccaa,
    {65 536, 131 072, 131 072, 131 073} → ccab,
    {65 536, 65 536, 131 072, 196 608} → cacc,
    {65 536, 65 536, 131 072, 131 073} → cacb,
    {65 536, 65 536, 65 536, 65 536} → caaa,
    {65 536, 65 536, 65 536, 65 537} → caab,
    {65 536, 65 536, 65 537, 131 073} → cabc,
    {65 536, 65 536, 65 537, 65 537} → caba,
    {65 536, 65 536, 65 537, 65 538} → cabb,
    {65 536, 131 072, 196 608, 196 609} → cccb,
    {65 536, 131 072, 131 073, 196 609} → ccbc,
    {65 536, 131 072, 131 073, 131 074} → ccbb,
    {65 536, 131 072, 131 073, 131 073} → ccba,
    {65 536, 65 537, 131 073, 196 609} → cbcc,
    {65 536, 65 537, 131 073, 131 073} → cbca,
    {65 536, 65 537, 65 538, 65 539} → cbbb,
    {65 536, 65 537, 65 538, 65 538} → cbba,
    {65 536, 65 537, 65 537, 131 073} → cbac,
    {65 536, 65 537, 65 537, 65 538} → cbab,
    {65 536, 65 537, 65 537, 65 537} → cbaa};

  Clear[constrFu, ordinalIndexForCumulIntegerList4];
constrFu[n_] := (ordinalIndexForCumulIntegerList4[#] :=
       Position[allCumulIntegersList4[[n]], #][[1, 1]]) & /@
    allCumulIntegersList4[[n]];

  constrFu[#] & /@ Range[Length[allCumulIntegersList4]];

  Clear[constructFu, tryProperExtensionVis];
constructFu[n_] := (tryProperExtensionVis[#, index_] :=
       €reducedWordListSuff4Add4ToExpressions[[
         ordinalIndexForCumulIntegerList4[#], 2, index]] /.
        €permRule[n]) & /@ allCumulIntegersList4[[n]];

  constructFu[#] & /@ Range[Length[allCumulIntegersList4]];

  Clear[constructFuVisWholeList, tryProperExtensionVisList];
constructFuVisWholeList[n_] := (tryProperExtensionVisList[#] :=
       €reducedWordListSuff4Add4ToExpressions[[
         ordinalIndexForCumulIntegerList4[#], 2]] /.
        €permRule[n]) & /@ allCumulIntegersList4[[n]];
```

```
constructFuVisWholeList[#] & /@
   Range[Length[allCumulIntegersList4]];

  Clear[tryProperExtension];
tryProperExtension[x_, index_] :=
   (tryProperExtensionVis[x, index] /.
     €ruleSymbolsToCumulIntegerLists);
tryProperExtension[x_] := (tryProperExtensionVisList[x] /.
     €ruleSymbolsToCumulIntegerLists);

  Clear[lengthOfListOfCatenableCumIntLis];
lengthOfListOfCatenableCumIntLis[x_List] :=
   Length[€reducedWordListSuff4Add4ToExpressions[[
     ordinalIndexForCumulIntegerList4[x], 2]]];

  Clear[abcRuleForList];
abcRuleForList[{a_, a_, a_, b_}] := {a → "a", b → "b",
    Complement[{"a", "b", "c"}, {a, b}][[1]] → "c"};
abcRuleForList[{a_, a_, b_, a_}] := {a → "a", b → "b",
    Complement[{"a", "b", "c"}, {a, b}][[1]] → "c"};
abcRuleForList[{a_, a_, b_, b_}] := {a → "a", b → "b",
    Complement[{"a", "b", "c"}, {a, b}][[1]] → "c"};
abcRuleForList[{a_, a_, b_, c_}] :=
   {a → "a", b → "b", c → "c"} /; Sort[{a, b, c}] === {"a", "b", "c"};
abcRuleForList[{a_, b_, a_, a_}] := {a → "a", b → "b",
    Complement[{"a", "b", "c"}, {a, b}][[1]] → "c"};
abcRuleForList[{a_, b_, a_, c_}] :=
   {a → "a", b → "b", c → "c"} /; Sort[{a, b, c}] === {"a", "b", "c"};
abcRuleForList[{a_, b_, b_, b_}] := {a → "a", b → "b",
    Complement[{"a", "b", "c"}, {a, b}][[1]] → "c"};
abcRuleForList[{a_, b_, b_, c_}] :=
   {a → "a", b → "b", c → "c"} /; Sort[{a, b, c}] === {"a", "b", "c"};
abcRuleForList[{a_, b_, c_, a_}] :=
   {a → "a", b → "b", c → "c"} /; Sort[{a, b, c}] === {"a", "b", "c"};
abcRuleForList[{a_, b_, c_, b_}] :=
   {a → "a", b → "b", c → "c"} /; Sort[{a, b, c}] === {"a", "b", "c"};
abcRuleForList[{a_, b_, c_, c_}] :=
   {a → "a", b → "b", c → "c"} /; Sort[{a, b, c}] === {"a", "b", "c"};)
```